

# FROM STUDENT TO TEACHER: A COGNITIVELY-GROUNDED FRAMEWORK FOR LLM ADAPTATION

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Large Language Model (LLM) adaptation is typically performed through monolithic optimization that simultaneously entangles knowledge consolidation, task alignment and generalization. In contrast, decades of cognitive science show that robust human expertise develops through temporally separated phases of learning, mastery and teaching. We introduce Learn-Master-Teach Tuning (LMT<sup>2</sup>), a cognitively grounded framework that structures LLM adaptation into staged phases mirroring the human student-to-teacher lifecycle. As a proof of concept, we instantiate STRIDE, a miniature representative three-stage pipeline. Across twelve benchmarks spanning commonsense, mathematics, logic, social, coding and medical reasoning, STRIDE consistently outperforms strong coupled baselines on Qwen2.5 models. Ablation studies show that the gains arise primarily from temporal separation itself, rather than from any specific optimization technique. Our results demonstrate that cognitive science offers actionable design principles for LLM training. We release the full LMT<sup>2</sup> blueprint as a roadmap for future work on cognitively aligned and robust model adaptation.

## 1 INTRODUCTION

Consider how a medical student becomes a surgeon. Training follows a carefully staged progression: years of foundational study (*Learning*), supervised practice with deliberate refinement (*Mastery*) and only after achieving expertise does one teach others (*Teaching*). This temporal structure reflects deep principles about robust expertise acquisition documented across cognitive science Ericsson et al. (1993); Chi & Wylie (2014). Yet LLM training violates these principles, expecting models to simultaneously consolidate knowledge, adapt to tasks and generalize across domains within monolithic optimization loops.

This paper addresses a central HCAIR question (see Table 1 for detailed workshop alignment):

*Can temporal separation of learning objectives - the staged progression from knowledge acquisition through mastery to teaching - provide principled inductive biases for LLM adaptation that produce more effective, robust and genuinely specialized systems?*

**The Student-to-Teacher Lifecycle.** Multiple lines of cognitive science research converge on the same insight: human expertise develops through distinct, temporally-separated phases. The **Dreyfus model** Dreyfus & Dreyfus (1980) describes progression from novice (rule-following) through competent (planning) to expert (intuitive mastery). **Anderson’s ACT-R theory** Anderson (1982) explains how declarative knowledge transforms into procedural expertise through skill compilation. **Complementary Learning Systems theory** McClelland et al. (1995) reveals how the brain uses dual memory systems i.e., hippocampal rapid learning is followed by neocortical consolidation, to avoid catastrophic forgetting. These theories independently predict that effective learning requires temporal separation of distinct objectives.

We synthesize these insights into a three-phase lifecycle:

- **Learning:** Novices acquire foundational knowledge through scaffolded instruction, building stable mental models before attempting complex application.

Table 1: HCAIR Workshop Themes Alignment Map to Cognitive Principles & LMT<sup>2</sup> Operationalization

| Theme                                                             | Cognitive Theory Inv.                                   | Operationalization in LMT <sup>2</sup> with STRIDE as PoC                                                                                                                                                  | Sec.   |
|-------------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Explicit modeling of human knowledge & capabilities               | Student-to-Teacher Lifecycle (Dreyfus, ACT-R, CLS)      | <b>Framework:</b> Unifies 11 developmental stages into a cohesive pipeline; <b>STRIDE:</b> Implements representative stages (Assimilation, Accommodation, Teaching) to mirror human expertise acquisition. | §3, §4 |
| Combining neural and symbolic methods inspired by human cognition | Equilibration (Piaget) (Assimilation vs. Accommodation) | <b>Temporal Separation:</b> Decouples implicit neural consolidation (Stage I) from explicit symbolic alignment (Stage II) to resolve the stability-plasticity dilemma and prevent gradient interference.   | §4.2   |
| Incorporating human teaching & correction into learning processes | Mastery Learning (Bloom) ZPD (Vygotsky)                 | <b>Stages 4, 5:</b> Enforces prerequisite mastery via interactive refinement and feedback-driven practice before ensuring target tasks fall within the model’s Zone of Proximal Development.               | §4.4   |
| Using human cognitive models for interpretability                 | Protégé Effect Metacognition (Flavell)                  | <b>Teaching Phase:</b> Forces the model to articulate consolidated knowledge to “student” models, making internal representations explicit and interpretable through the act of teaching.                  | §3.3   |
| Structured approaches to causal & human-inspired AI reasoning.    | Complementary Learning Systems (McClelland)             | <b>Dual-System Optimization:</b> Separates rapid episodic learning (Stage I) from gradual schematic consolidation (Stage II/VI) to build robust causal structures without catastrophic forgetting.         | §2     |

- **Mastery:** Practitioners refine their knowledge through deliberate practice, aligning general principles with specific contexts and developing fluent, flexible expertise.
- **Teaching:** Experts consolidate their understanding by articulating it to others (the protégé effect demonstrates that teaching produces deeper and more durable knowledge than passive review Chase et al. (2009); Roscoe & Chi (2007)).

Critically, these phases are not pursued simultaneously. In fact, the Cognitive load theory Sweller (1988) predicts that concurrent pursuit of conflicting objectives degrades learning. Attempting to skip phases or conflate objectives produces fragile, rote performance rather than flexible expertise.

**Our Contributions:** We make three contributions that validate temporal separation as a design principle for LLM adaptation:

1. **The LMT<sup>2</sup> Framework** (§3): A comprehensive cognitive framework comprising **nine stages across three phases** mirroring human expertise development: *Learning* (4 stages), *Mastery* (3 stages) and *Teaching* (2 stages). Each stage is grounded in cognitive science with detailed implementation methodologies in the Appendix, providing a roadmap for future research.
2. **Empirical Validation via STRIDE** (§4, §5): Our primary contribution, we implement Staged Temporal Reasoning for Intelligent Developmental Evolution (STRIDE), a three-stage pipeline embodying the core perspective of each LMT<sup>2</sup> phase. On 12 benchmarks, STRIDE demonstrates that temporal separation itself, and not specific techniques, drives improvement, validating the core cognitive principle.
3. **Cognitive Science as Design Principle:** We demonstrate that cognitive science provides actionable AI training principles, not merely post-hoc explanations. This methodological contribution advances the HCAIR goal of bridging human cognition and AI reasoning.

**Paper Organization.** Section 2 establishes the cognitive science foundations such as the Dreyfus model, ACT-R, Complementary Learning Systems, Bloom’s mastery learning, Piaget’s equilibration and the protégé effect, that motivate our design. Section 3 briefly presents the complete LMT<sup>2</sup> framework with all nine stages. Section 4 details the STRIDE proof-of-concept implementation. Section 5 provides empirical validation with results and ablation studies. Section 6 presents the concluding remarks.

## 2 COGNITIVE FOUNDATIONS: WHY TEMPORAL SEPARATION?

The LMT<sup>2</sup> framework is grounded in converging evidence from cognitive science, developmental psychology, neuroscience and educational theory. A striking pattern emerges: **fifteen theories from different research traditions independently predict that effective learning requires temporal separation of distinct objectives**. This convergence is not coincidental, it reflects a fundamental

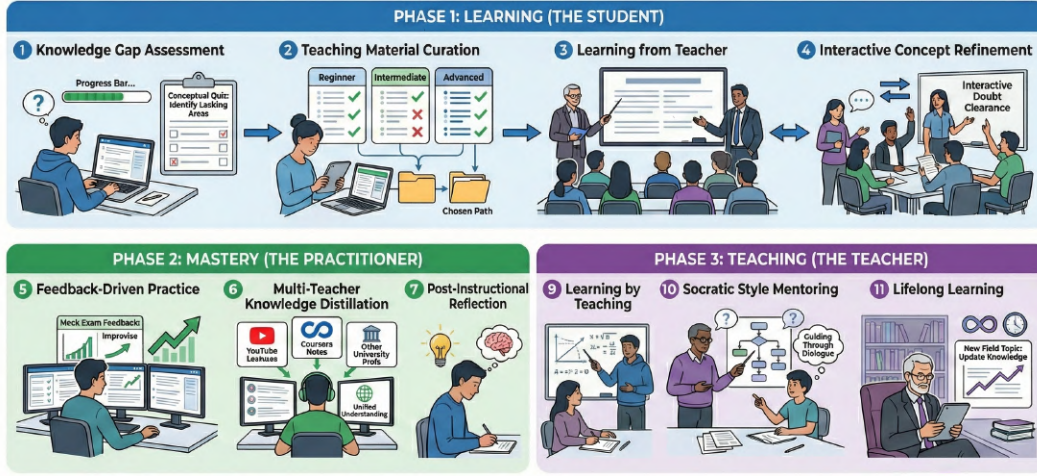


Figure 1: The complete human student to teacher life-cycle providing inspiration for our proposed LMT<sup>2</sup> framework which structures LLM adaptation through nine stages across three phases, mirroring the human expertise development.

property of how complex adaptive systems avoid interference when pursuing multiple goals. Table 2 presents a non-exhaustive mapping of these theories to their operationalization in LMT<sup>2</sup>. For each theory, we identify the core principle that motivates temporal separation and specify how this principle is realized in our framework. Extended theoretical grounding, including detailed analysis of each theory, is provided in Appendix A.

**Unified Insight: Temporal Separation.** The convergent prediction across all theories is that *effective learning requires temporal separation of learning objectives*. Whether framed as phase transitions (Dreyfus, Fitts-Posner), memory system complementarity (CLS), equilibration cycles (Piaget), or load management (CLT), the underlying principle remains consistent: **attempting to optimize multiple conflicting objectives simultaneously produces suboptimal “compromise” solutions**. This generates a testable prediction: structuring LLM adaptation into temporally-separated phases-Learning, Mastery and Teaching-should outperform monolithic optimization. STRIDE validates this prediction through a minimal three-stage implementation, with results reported in Section 5.

### 3 LEARN-MASTER-TEACH TUNING FRAMEWORK

LMT<sup>2</sup> (Learn-Master-Teach Tuning) operationalizes the cognitive principles from Section 2 through a comprehensive nine-stage framework for LLM adaptation, organized into three phases that mirror the complete student-to-teacher lifecycle of human expertise development. This section presents the visionary framework with detailed implementation methodologies for each stage in the respective appendices.

#### 3.1 PHASE 1: LEARNING (THE STUDENT)

The Learning phase corresponds to the *novice/advanced beginner* stages in the Dreyfus model Dreyfus & Dreyfus (1980), where learners acquire declarative knowledge through scaffolded instruction Vygotsky (1978). This phase comprises four stages:

**Stage 1: Knowledge Gap Assessment.** Before effective learning can begin, the learner must recognize what they do not know-transitioning from *unconscious incompetence* to *conscious incompetence*. For LLMs, this involves probing current capabilities to identify knowledge gaps that training should address (Appendix C.1).

Table 2: Cognitive Foundations Table

| Theory/Model                                          | Core Principle                                                                              | Operationalization in LMT <sup>2</sup> /STRIDE                                                               | Phase |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-------|
| <b>Developmental &amp; Expertise Acquisition</b>      |                                                                                             |                                                                                                              |       |
| Dreyfus Model                                         | 5-stage progression: Novice → Advanced Beginner → Competent → Proficient → Expert           | 11-stage LMT <sup>2</sup> progression mirrors expertise development; STRIDE validates core phases            | 1-3   |
| Fitts-Posner Model                                    | Cognitive → Associative → Autonomous phases of skill acquisition                            | Phase 1 (cognitive learning) → Phase 2 (practice/association) → Phase 3 (autonomous teaching)                | 1-3   |
| ACT-R Theory                                          | Declarative knowledge compiles into procedural fluency through practice                     | Stages 1-4 (declarative acquisition) → Stages 5-8 (procedural compilation via practice)                      | 1-2   |
| <b>Learning Theory &amp; Developmental Psychology</b> |                                                                                             |                                                                                                              |       |
| Bloom’s Learning                                      | Mastery Master prerequisites before advancing to dependent concepts                         | Stage completion gating; foundation consolidation before task-specific refinement (Stage I → II)             | 1     |
| Piaget’s Equilibration                                | Assimilation (integrate new info) + Accommodation (modify structures) cycles                | Stage I = Assimilation; Stage II = Accommodation; temporal separation prevents stability-plasticity conflict | 1-2   |
| Vygotsky’s ZPD                                        | Learning effective only within zone between current and potential ability                   | ZPD-calibrated question matrices (Stage 5); scaffolded materials matched to model capability (Stage 2)       | 1-2   |
| Cognitive Load Theory                                 | Simultaneous objectives cause interference; sequential presentation improves learning       | Temporal separation eliminates gradient interference; each stage optimizes single objective                  | 1-3   |
| <b>Neuroscience &amp; Memory Systems</b>              |                                                                                             |                                                                                                              |       |
| Complementary Learning Systems                        | Fast hippocampal encoding + slow neocortical consolidation prevents catastrophic forgetting | Stage 7: Daytime scaling (fast episodic) + Nighttime scaling (slow procedural consolidation)                 | 2     |
| Neuroplasticity                                       | Experience-dependent synaptic changes underlie learning                                     | LoRA weight adaptation as computational neuroplasticity; staged updates preserve prior learning              | 1-3   |
| <b>Pedagogy &amp; Instructional Design</b>            |                                                                                             |                                                                                                              |       |
| Bruner’s Scaffolding                                  | Temporary support gradually withdrawn as competence develops                                | Faded scaffolding in Stages 2-3; teacher guidance reduces as student model improves                          | 1     |
| Expertise Reversal Effect                             | Novices and experts require different instructional approaches                              | Adaptive teaching in Stage 2; materials calibrated to current model capability                               | 1     |
| protégé Effect                                        | Teaching others deepens the teacher’s own understanding                                     | Stage 9: Learning by Teaching; DPO training based on student feedback                                        | 3     |
| Socratic Method                                       | Questioning-based learning promotes deeper understanding than direct instruction            | Stage 4: Interactive refinement; Stage 10: Socratic mentoring                                                | 1, 3  |
| <b>Metacognition &amp; Skill Acquisition</b>          |                                                                                             |                                                                                                              |       |
| Metacognition                                         | Self-monitoring and self-correction improve reasoning quality                               | Stage 7: Post-Instructional Reflection; self-rewarding quality assessment                                    | 2     |
| Deliberate Practice                                   | Structured practice with feedback produces expertise faster than unstructured repetition    | Stage 5: Feedback-Driven Practice; iterative refinement with corrective guidance                             | 2     |
| Testing Effect                                        | Retrieval practice enhances long-term retention more than passive review                    | Stage 5: Practice via MCQ questioning; Stage 8: Examination for validation                                   | 2     |

**Stage 2: Teaching Material Curation.** Following the expertise reversal principle Kalyuga et al. (2003), pedagogical content must be appropriately structured for the learner’s current level. This stage curates and adapts training materials to match the model’s Zone of Proximal Development (Appendix C.2).

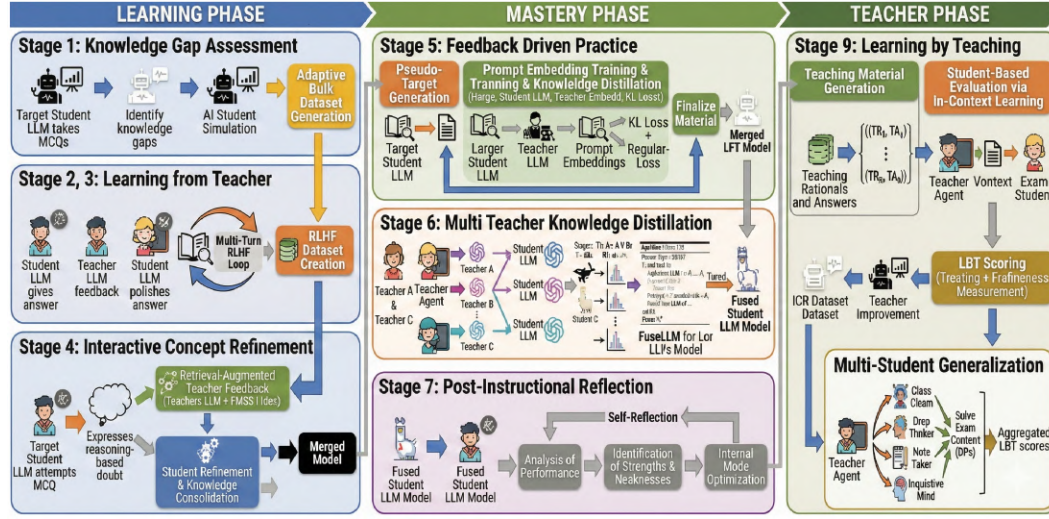
**Stage 3: Learning from Teacher.** The core knowledge acquisition stage, where the student model receives instruction from data curated by teacher models. This corresponds to Piaget’s assimilation phase Piaget (1952), where new information is integrated into emerging cognitive structures.

**Stage 4: Interactive Concept Refinement.** Drawing from Socratic pedagogy Reich (2003) and interactive learning theory Chi & Wylie (2014), this stage enables interactive clarification and doubt resolution, ensuring foundational concepts are properly understood before proceeding to application (Appendix C.3).

### 3.2 PHASE 2: MASTERY (THE PRACTITIONER)

The Mastery phase corresponds to the *competent/proficient* stages in the Dreyfus model, where practitioners develop flexible expertise through deliberate practice Ericsson et al. (1993). In ACT-R terms Anderson (1982), this phase transforms declarative knowledge into procedural skill through skill compilation. This phase comprises four stages:



Figure 2: High-Level Overview of the Implementation Blueprint for LMT<sup>2</sup>

**Stage 5: Feedback-Driven Practice.** Following Bloom’s mastery learning Bloom (1968), this stage provides iterative practice with corrective feedback. The model applies learned knowledge to increasingly challenging problems, receiving guidance that addresses specific weaknesses (Appendix D.1).

**Stage 6: Multi-Teacher Knowledge Distillation.** Drawing from Complementary Learning Systems theory McClelland et al. (1995), this stage consolidates knowledge from multiple expert sources. Unlike the rapid hippocampal learning in Phase 1, this stage emphasizes gradual neocortical consolidation that integrates diverse expertise (Appendix D.2).

**Stage 7: Post-Instructional Reflection.** Metacognitive reflection Flavell (1979) enables the learner to evaluate their own understanding and identify remaining gaps. This stage implements test-time reflection and self-assessment procedures. This is followed by an optional *examination* (stage 8) which is a competence assessment stage that verifies prerequisite mastery before advancing to teaching. This operationalizes the testing effect Roediger & Karpicke (2006), where retrieval practice enhances long-term retention (Appendix D.3).

### 3.3 PHASE 3: TEACHING (THE TEACHER)

The Teaching phase corresponds to the *expert* stage in the Dreyfus model, leveraging the protégé effect Chase et al. (2009); Roscoe & Chi (2007) for deep knowledge consolidation. Teaching others produces deeper understanding than passive review because articulating knowledge requires explicit organization of implicit understanding. This phase comprises three stages:

**Stage 9: Learning by Teaching.** The core teaching stage, where the expert model teaches student models. Research in educational psychology Biswas et al. (2005); Ning et al. (2024b) demonstrates that explaining concepts to others forces knowledge reorganization and identifies gaps in understanding. This is sometimes normally followed by a *Socratic-Style Mentoring* (stage 10) which involves going beyond direct instruction, to enable the expert to guide students through questioning rather than telling. This Socratic approach Reich (2003) also deepens the mentor’s own understanding while developing student autonomy. (Appendix E.1)

**Stage 11: Lifelong Learning** The cycle continues as experts encounter novel challenges that require returning partially to the learning phase. This stage implements continual learning procedures that integrate new knowledge without catastrophic forgetting Kumaran et al. (2016).

## 4 STRIDE: A PROOF-OF-CONCEPT IMPLEMENTATION

To validate the core principle of our proposed LMT<sup>2</sup> framework, that temporal separation of learning objectives produces more effective adaptation, we present STRIDE (Staged Temporal Reasoning for Intelligent Developmental Evolution), a miniature three-stage pipeline that implements one representative perspective from each LMT<sup>2</sup> phase. This proof-of-concept demonstrates that even a minimal instantiation of the student-to-teacher lifecycle produces significant improvements over monolithic baselines. This section presents the technical details; full implementation methodologies for STRIDE is detailed in Appendix B. Table 3 presents the explicit mapping between STRIDE’s three stages and the corresponding LMT<sup>2</sup> phases, along with the cognitive theories each stage validates. This mapping demonstrates how STRIDE is a *minimal viable instantiation* of the cognitively-principled LMT<sup>2</sup> framework.

Table 3: LMT<sup>2</sup> → STRIDE Stage Mapping

| LMT <sup>2</sup> Phase                                             | STRIDE Stage                           | What It Validates                                                                                                  | Key Cognitive Theories                                                            |
|--------------------------------------------------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Phase 1: Learning</b><br>(Stages 1-4)<br><i>The Student</i>     | Stage I:<br>Foundation Consolidation   | Building stable domain representations before task-specific adaptation; knowledge gap identification               | Bloom’s Mastery Learning, Vygotsky’s ZPD, Bruner’s Scaffolding                    |
| <b>Phase 2: Mastery</b><br>(Stages 5-8)<br><i>The Practitioner</i> | Stage II:<br>Knowledge-Task Alignment  | Adapting general knowledge to task requirements through Fusion VAE; equilibration between stability and plasticity | Piaget’s Equilibration, ACT-R (skill compilation), Complementary Learning Systems |
| <b>Phase 3: Teaching</b><br>(Stages 9-11)<br><i>The Teacher</i>    | Stage III:<br>Task-Specific Refinement | Deploying learned knowledge flexibly; test-time adaptation strategies selected via RL                              | protégé Effect, Deliberate Practice, Metacognition                                |

### 4.1 PROBLEM SETTING AND NOTATION

Let  $f_\theta$  denote a pretrained LLM with frozen base parameters  $\theta_0 \in \mathbb{R}^P$ . We consider adaptation through Low-Rank Adaptation (LoRA) Hu et al. (2022b), where task-specific behavior is achieved by learning low-rank adapter matrices. For a weight matrix  $W_0 \in \mathbb{R}^{d_1 \times d_2}$ , LoRA introduces,  $W = W_0 + \Delta W = W_0 + BA^\top$ ,  $A \in \mathbb{R}^{r \times d_1}, B \in \mathbb{R}^{d_2 \times r}$  where  $r \ll \min(d_1, d_2)$  is the adapter rank. The complete adapter parameter vector  $\theta^a \in \mathbb{R}^D$  concatenates all LoRA matrices across model layers. Given a distribution of training tasks  $p(\mathcal{T})$  and a target task  $\mathcal{T}_*$  with unlabeled examples  $\mathcal{D}_{\mathcal{T}_*} = \{x_i\}_{i=1}^N$ , our goal is to produce adapted parameters  $\theta_*^a$  that maximize performance on  $\mathcal{T}_*$ .

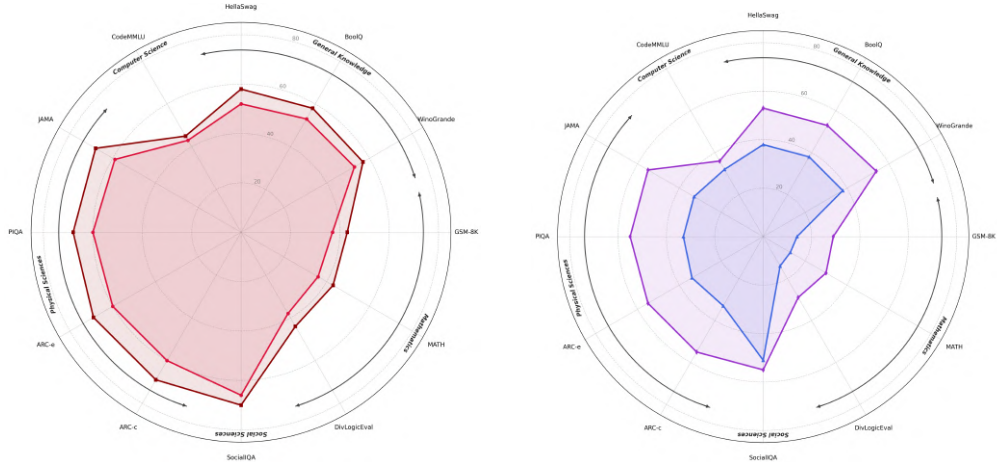


Figure 3: Radar Chart demonstrating performance impact of STRIDE on Base Model - (a) Qwen2.5-1.5B-Instruct (left: Base Model, after STRIDE) and (b) Qwen2.5-0.5B-Instruct (right: Base Model, after STRIDE)

## 4.2 STAGE I: FOUNDATION CONSOLIDATION (ASSIMILATION)

The first stage consolidates foundational knowledge by learning a prompt-to-parameters mapping, paralleling Piaget’s assimilation phase where new information is integrated into stable structures. We train a decoder (Hyper-Convolutional Parameter Generator)  $\mathcal{G}_\phi$  that generates LoRA parameters from task prompt embeddings:  $\theta_{\text{gen}}^a = \mathcal{G}_\phi(\mathbf{C})$ , where  $\mathbf{C} = \text{Enc}(\{p_i\}_{i=1}^N) \in \mathbb{R}^{N \times L \times C}$ . Here,  $\{p_i\}$  are representative prompts from the target domain,  $\text{Enc}(\cdot)$  is a frozen text encoder (Sentence-BERT) and  $(N, L, C)$  denote the number of prompts, sequence length and embedding dimension. The generator is trained via supervised reconstruction on LoRA checkpoints collected from task-specific fine-tuning. For each training task  $\mathcal{T}_i$ , we perform standard LoRA fine-tuning to obtain ground-truth parameters  $\theta_{\mathcal{T}_i}^{a,*}$ , then train  $\mathcal{G}_\phi$  to minimize:  $\mathcal{L}_{\text{consolidation}} = \mathbb{E}_{\mathcal{T}_i \sim p(\mathcal{T})} [\|\mathcal{G}_\phi(\mathbf{C}_{\mathcal{T}_i}) - \theta_{\mathcal{T}_i}^{a,*}\|^2]$ . This stage produces a generator that encodes cross-task regularities without task-specific optimization pressure, establishing the “stable structures” that cognitive theory identifies as prerequisites for productive accommodation. Full information is provided in Appendix B.2

## 4.3 STAGE II: KNOWLEDGE-TASK ALIGNMENT (ACCOMMODATION)

The second stage aligns consolidated knowledge with target task semantics through a conditional Variational Autoencoder (VAE), paralleling Piaget’s accommodation phase where mental structures are modified to fit new information. In this stage, first, following task-vector formulations Ilharco et al. (2023), we define  $\mathbf{v}_{\mathcal{T}_*} = \mathbf{z}_{\mathcal{T}_*}^{\text{ft}} - \mathbf{z}^{(0)}$ , where  $\mathbf{z}^{(0)}$  and  $\mathbf{z}_{\mathcal{T}_*}^{\text{ft}}$  denote last-layer representations of the pretrained and task-adapted models, respectively. Then, given generated parameters  $\theta_{\text{gen}}^a$  and task vector  $\mathbf{v}_{\mathcal{T}_*}$ , we train a conditional variational auto-encoder where, the encoder models

$$q_\phi(\mathbf{z} \mid \theta_{\text{gen}}^a, \mathbf{v}_{\mathcal{T}_*}) = \mathcal{N}(\mu_\phi([\theta_{\text{gen}}^a; \mathbf{v}_{\mathcal{T}_*}]), \sigma_\phi^2([\theta_{\text{gen}}^a; \mathbf{v}_{\mathcal{T}_*}])\mathbf{I}), \quad (1)$$

and the decoder reconstructs aligned parameters:  $\theta_{\text{aligned}}^a = \text{Dec}_\psi(\mathbf{z}, \mathbf{v}_{\mathcal{T}_*})$ ,  $\mathbf{z} \sim q_\phi(\mathbf{z} \mid \cdot)$ . The alignment objective is

$$\mathcal{L}_{\text{align}} = \mathbb{E}_{q_\phi} [\|\theta_{\text{gen}}^a - \theta_{\text{aligned}}^a\|^2] + \lambda_{\text{KL}} D_{\text{KL}}(q_\phi(\mathbf{z} \mid \cdot) \parallel \mathcal{N}(0, \mathbf{I})) \quad (2)$$

Full information is provided in Appendix B.3

## 4.4 STAGE III: TASK-SPECIFIC REFINEMENT (ZPD-BOUNDED ADAPTATION)

The final stage optimizes for task performance, operating within the model’s (now-expanded) Zone of Proximal Development. Given aligned parameters  $\theta_{\text{aligned}}^a$  from Stage II, Stage III performs lightweight, task-conditioned specialization. Rather than executing a fixed gradient rule, refinement is cast as a decision-making problem over a family of adaptation strategies:  $\mathcal{A} = \{\text{TTL}, \text{LoRA}, \text{TTS}, \text{Latent}, \dots\}$ , where each  $\alpha \in \mathcal{A}$  specifies a bounded inference-time transformation. The task-specialized adapter is produced via:  $\theta_*^a = \theta_{\text{aligned}}^a \oplus \text{Update}(\theta_{\text{aligned}}^a, \alpha)$ . STRIDE parameterizes a refinement policy  $\pi_\phi$  over  $\mathcal{A}$  and optimizes it through reinforcement learning:  $\alpha^* = \arg \max_{\alpha \in \mathcal{A}} \mathbb{E}_{\alpha \sim \pi_\phi(\cdot \mid \tau)} [R(\tau, \alpha)]$ , where  $\tau$  denotes the task context and  $R$  evaluates refinement utility. Crucially, RL supervises the *adaptation behavior*—how to adapt—rather than directly optimizing task outputs. Unlike prolonged online fine-tuning, STRIDE constrains refinement to bounded inference-time steps, yielding efficient task specialization. Extended discussion is provided in Appendix B.4.

## 5 EMPIRICAL VALIDATION

We evaluate STRIDE on diverse reasoning tasks, testing both task performance and the cognitive predictions that motivated our design: does temporal separation reduce gradient interference, produce sparser adaptation and yield task-specific rather than generic solutions?

**Models.** We evaluate Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct Qwen et al. (2025). All base model parameters are frozen and adaptation is performed exclusively via LoRA adapters. Experiments were conducted on a single Ubuntu 22.04.1 HPC node with an AMD EPYC 8434P CPU (48 cores/96 threads), 256 GB RAM and four NVIDIA RTX A6000 GPUs (48 GB VRAM each).

| Domain                       | Physical Sciences |             |                  |                |             |
|------------------------------|-------------------|-------------|------------------|----------------|-------------|
| Dataset                      | ARC-c             | ARC-e       | PIQA             | JAMA           | Avg         |
| <i>Qwen2.5-1.5B-Instruct</i> |                   |             |                  |                |             |
| Base Model                   | 71.5              | 83.0        | 45.8             | 38.9           | 59.8        |
| Union Train LoRA             | 63.2              | 73.9        | 47.8             | 34.7           | 54.9        |
| ABMLL                        | 69.9              | 83.2        | 54.3             | 28.5           | 59.0        |
| MAML-en-LLM                  | 66.0              | 84.3        | 68.1             | 32.5           | 62.7        |
| <b>STRIDE (proposed)</b>     | <b>73.7</b>       | <b>88.4</b> | <b>70.7</b>      | <b>41.5</b>    | <b>68.6</b> |
| <i>Qwen2.5-0.5B-Instruct</i> |                   |             |                  |                |             |
| Base Model                   | 38.3              | 54.8        | 16.6             | 23.8           | 33.4        |
| Union Train LoRA             | 39.7              | 47.4        | 51.1             | 29.9           | 42.0        |
| ABMLL                        | 37.6              | 54.4        | 37.6             | 16.8           | 36.6        |
| MAML-en-LLM                  | 47.7              | 63.7        | 67.7             | 26.4           | 51.4        |
| SeRL                         | 22.4              | 24.0        | <b>96.7</b>      | 12.4           | 38.9        |
| <b>STRIDE (proposed)</b>     | <b>55.5</b>       | <b>74.7</b> | 60.1             | <b>31.2</b>    | <b>55.4</b> |
| Domain                       | General Knowledge |             |                  | Social Science |             |
| Dataset                      | BoolQ             | HellaSwag   | WinoGrande       | SocialIQA      | Avg         |
| <i>Qwen2.5-1.5B-Instruct</i> |                   |             |                  |                |             |
| Base Model                   | 56.3              | 50.9        | 50.6             | 65.9           | 52.6        |
| Union Train LoRA             | 55.1              | 48.9        | <b>61.3</b>      | 51.4           | 54.2        |
| ABMLL                        | <b>63.2</b>       | 51.1        | 52.9             | 66.3           | 58.4        |
| MAML-en-LLM                  | 58.7              | <b>59.3</b> | 56.8             | 68.7           | <b>60.9</b> |
| <b>STRIDE (proposed)</b>     | 58.8              | 57.2        | 57.3             | <b>69.5</b>    | 60.7        |
| <i>Qwen2.5-0.5B-Instruct</i> |                   |             |                  |                |             |
| Base Model                   | 37.0              | 26.5        | 50.2             | 50.8           | 37.9        |
| Union Train LoRA             | 14.7              | 26.3        | 50.5             | 39.5           | 32.8        |
| ABMLL                        | <b>62.2</b>       | 26.5        | 34.5             | 53.1           | 44.1        |
| MAML-en-LLM                  | 46.2              | 36.3        | 50.1             | 54.9           | 46.9        |
| SeRL                         | 44.7              | 25.0        | 49.6             | 33.0           | 38.1        |
| <b>STRIDE (proposed)</b>     | 58.7              | <b>48.3</b> | <b>52.8</b>      | <b>55.1</b>    | <b>53.7</b> |
| Domain                       | Mathematics       |             | Computer Science |                |             |
| Dataset                      | GSM-8K            | MATH        | DivLogicEval     | CodeMMLU       | Avg         |
| <i>Qwen2.5-1.5B-Instruct</i> |                   |             |                  |                |             |
| Base Model                   | 51.4              | 30.3        | 28.3             | 42.6           | 36.7        |
| Union Train LoRA             | 34.2              | 32.2        | 24.1             | 34.7           | 31.3        |
| ABMLL                        | 28.7              | 15.9        | 26.9             | 39.6           | 27.8        |
| MAML-en-LLM                  | 35.6              | 43.5        | 31.2             | 42.3           | 38.2        |
| <b>STRIDE (proposed)</b>     | <b>51.8</b>       | <b>46.9</b> | <b>31.4</b>      | <b>44.6</b>    | <b>43.6</b> |
| <i>Qwen2.5-0.5B-Instruct</i> |                   |             |                  |                |             |
| Base Model                   | 15.2              | 2.8         | 22.4             | 32.4           | 13.5        |
| Union Train LoRA             | 15.6              | 6.8         | 20.3             | 29.8           | 18.1        |
| ABMLL                        | 20.4              | 7.1         | 23.7             | 28.2           | 19.9        |
| MAML-en-LLM                  | 29.1              | <b>26.3</b> | 25.1             | 34.1           | 28.7        |
| SeRL                         | 24.0              | 24.7        | 27.4             | 26.3           | 25.6        |
| <b>STRIDE (proposed)</b>     | <b>30.3</b>       | 24.5        | <b>28.7</b>      | <b>35.6</b>    | <b>29.8</b> |

Table 4: Main results comparing STRIDE against base models and various adaptation methods.

**Datasets.** We use multiple datasets for evaluation spanning across domains such as ARC-challenge, ARC-easy, PIQA and JAMA for Physical Science, BoolQ HellaSwag and WinoGrande for General Knowledge, GSM-8K MATH and DivLogicEval for Mathematics, CodeMMLU for Computer Science and lastly SocialIQA for Social Science.

**Baselines.** We compare STRIDE against the following adaptation strategies: (i) *Union Train LoRA*, which trains a single LoRA adapter on the union of all training tasks; (ii) *MAML-en-LLM* Sinha et al. (2024) which is a meta-learning based adaptation method; (iii) *ABMLL* Zhang et al. (2025a) which is an amortized bayesian method for adaptation and (iv) *SeRL* Fang et al. (2025), which is an adaptation by self-evolution method. They have been chosen because methods such as *MAML-en-LLM* and *ABMLL*, despite being state-of-the-art and seminal works, fundamentally violate the



temporal separation principle for LLMs and couple the generalization, alignment and adaptation objectives together as a single bi-level optimization framework.

**Results.** Table 4 presents performance across all 12 benchmarks. STRIDE achieves consistent improvements across domains, with particularly strong gains on smaller models where parameter constraints make temporal separation most impactful. Notably, STRIDE outperforms baseline methods consistently which couple multiple objectives, validating our hypothesis that monolithic optimization produces suboptimal “compromise” solutions.

**Ablation Analysis.** Figure 4 shows the incremental contribution of each STRIDE stage on Qwen2.5-0.5B-Instruct. **Stage I (Foundation Consolidation)** provides the critical base—without stable domain representations, subsequent stages cannot effectively align or refine. **Stage II (Knowledge-Task Alignment)** builds substantially on Stage I, confirming Piaget’s prediction that accommodation requires prior assimilation. **Stage III (Task-Specific Refinement)** provides final task-specific specialization within the model’s expanded Zone of Proximal Development. Crucially, *stage skipping* (e.g., applying Stage III directly) produces minimal improvement, validating Bloom’s mastery learning principle that prerequisites must be satisfied before advancement.

**Validating Cognitive Predictions.** Our results support three key cognitive predictions: (1) *Temporal separation reduces interference*—sequential staging avoids the gradient conflicts inherent in coupled monolithic optimization; (2) *Prerequisite mastery enables subsequent learning*—Stage I consolidation is necessary for Stage II/III effectiveness; (3) *Staged approaches generalize better*—STRIDE’s out-of-domain performance (mathematics, medical reasoning) suggests temporal separation builds more transferable representations. Extended behavioral analysis in Appendix B.5 validates these predictions through layer activation patterns and LoRA weight sparsity analysis.

**Limitations.** *Implementation scope:* STRIDE validates only the core principle of LMT<sup>2</sup> but not that of the pipeline in its entirety. *Model scale:* Results on 0.5B-1.5B models; larger models may show different dynamics. *Benchmark scope:* English-only evaluation; multilingual generalization is untested. Despite these limitations, our results validate the *core principle* that temporal separation improves adaptation, motivating fuller LMT<sup>2</sup> implementation.

## 6 SUMMARY

We presented LMT<sup>2</sup>, a cognitively-grounded framework structuring LLM adaptation through nine stages across three phases—Learning, Mastery and Teaching—mirroring the student-to-teacher lifecycle of human expertise development. Through STRIDE, we validated the core principle: **temporal separation of learning objectives produces more effective adaptation than monolithic optimization**. This work contributes to the HCAIR vision by demonstrating that cognitive science provides actionable design principles for AI training. Our proof-of-concept validates the core principle of LMT<sup>2</sup>; with the whole pipeline itself offering rich research directions. The complete blueprint with detailed implementation methodologies (Appendices C, D, E) provides a roadmap for this visionary research agenda. We hope this work encourages deeper integration between cognitive science and AI development, advancing human-aligned artificial reasoning.

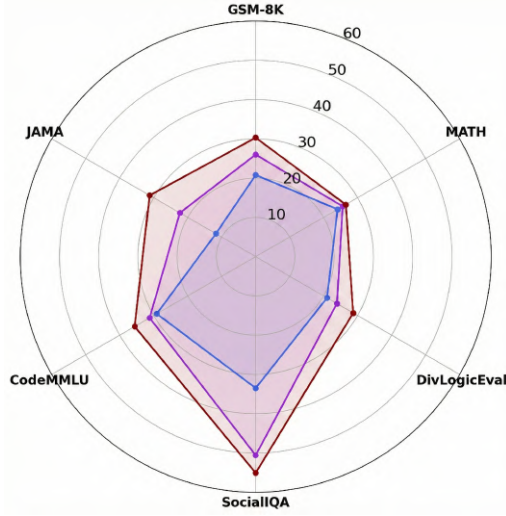


Figure 4: Ablation Study revealing the incremental effectiveness of each STRIDE stage with **Stage I**, **Stage II** and **Stage III**

## REFERENCES

- Shengnan An, Zexiong Ma, Zeqi Lin, Nanning Zheng, Jian-Guang Lou, and Weizhu Chen. Learning from mistakes makes llm better reasoner. *arXiv preprint arXiv:2310.20689*, 2023.
- John R Anderson. Acquisition of cognitive skill. *Psychological review*, 89(4):369–406, 1982.
- John R Anderson. *How can the human mind occur in the physical universe?* Oxford University Press, 2007.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language, 2019. URL <https://arxiv.org/abs/1911.11641>.
- Gautam Biswas, Krittaya Leelawong, Daniel Schwartz, Nancy Vye, et al. Learning by teaching: A new agent paradigm for educational software. *Applied artificial intelligence*, 19(3-4):363–392, 2005.
- James H Block. Mastery learning: Theory and practice. 1971.
- Benjamin S Bloom. Learning for mastery. *Evaluation comment*, 1(2):1–12, 1968.
- Rujikorn Charakorn, Edoardo Cetin, Yujin Tang, and Robert Tjarko Lange. Text-to-lora: Instant transformer adaption, 2025. URL <https://arxiv.org/abs/2506.06105>.
- Catherine C Chase, Doris B Chin, Marily A Oppezzo, and Daniel L Schwartz. Teachable agents and the protégé effect: Increasing the effort towards learning. *Journal of Science Education and Technology*, 18(4):334–352, 2009.
- Hanjie Chen, Zhouxiang Fang, Yash Singla, and Mark Dredze. Benchmarking large language models on answering and explaining challenging medical questions. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 3563–3599, Albuquerque, New Mexico, April 2025a. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.182. URL <https://aclanthology.org/2025.naacl-long.182/>.
- Wei Chen, Yuxuan Zhang, et al. Lepa: Latent-space error-based prompt augmentation for question answering. In *International Conference on Learning Representations*, 2025b.
- Xiaoyu Chen et al. Mcqg-srefine: Multiple choice question generation with self-refine. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- Micheline TH Chi and Ruth Wylie. The icap framework: Linking cognitive engagement to active learning outcomes. *Educational psychologist*, 49(4):219–243, 2014.
- Micheline TH Chi, Stephanie A Siler, Heisawn Jeong, Takeshi Yamauchi, and Robert G Hausmann. Learning from human tutoring. *Cognitive science*, 25(4):471–533, 2001.
- Tsz Ting Chung, Lemao Liu, Mo Yu, and Dit-Yan Yeung. Divlogiceval: A framework for benchmarking logical reasoning evaluation in large language models, 2025. URL <https://arxiv.org/abs/2509.15587>.
- Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In Jill Burstein, Christy Doran, and Tamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1300. URL <https://aclanthology.org/N19-1300/>.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018. URL <https://arxiv.org/abs/1803.05457>.

- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Allan Collins, John Seely Brown, and Susan E Newman. Cognitive apprenticeship: Making thinking visible. *American educator*, 15(3):6–11, 1989.
- David Dinucu-Jianu, Jakub Macina, Nico Daheim, Ido Hakimi, Iryna Gurevych, and Mrinmaya Sachan. From problem-solving to teaching problem-solving: Aligning llms with pedagogy using reinforcement learning. *arXiv preprint arXiv:2505.15607*, 2025.
- Stuart E Dreyfus and Hubert L Dreyfus. *A five-stage model of the mental activities involved in directed skill acquisition*. California Univ Berkeley Operations Research Center, 1980.
- K Anders Ericsson, Ralf T Krampe, and Clemens Tesch-Römer. The role of deliberate practice in the acquisition of expert performance. *Psychological review*, 100(3):363, 1993.
- Wenkai Fang, Shunyu Liu, Yang Zhou, et al. SeRL: Self-play reinforcement learning for large language models with limited data. In *Advances in Neural Information Processing Systems*, 2025.
- Paul M Fitts and Michael I Posner. *Human performance*. Brooks/Cole, 1967.
- John H Flavell. Metacognition and cognitive monitoring: A new area of cognitive–developmental inquiry. *American psychologist*, 34(10):906, 1979.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. *arXiv preprint arXiv:2306.08543*, 2023.
- Tao Gui et al. Learn-by-interact: Synthesizing agent data from environment interactions. *arXiv preprint arXiv:2410.10384*, 2024.
- Changyi He, Yifu Ding, Jinyang Guo, Ruihao Gong, Haotong Qin, and Xianglong Liu. DA-KD: Difficulty-aware knowledge distillation for efficient large language models. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=NCYBdRCpwl>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022a.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022b.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations*, 2023.
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Grave, and Armand Joulin. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*, 2022.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- Slava Kalyuga, Paul Ayres, Paul Chandler, and John Sweller. The expertise reversal effect. *Educational psychologist*, 38(1):23–31, 2003.

- Gyeongman Kim, Doohyuk Jang, and Eunho Yang. Promptkd: Distilling student-friendly knowledge for generative language models via prompt tuning, 2024. URL <https://arxiv.org/abs/2402.12842>.
- Bryan Kolb and Ian Q Whishaw. *Fundamentals of human neuropsychology*. Worth Publishers, 6th edition, 2009.
- Kalle Kujanpää, Pekka Marttinen, Harri Valpola, and Alexander Ilin. Efficient knowledge injection in llms via self-distillation, 2025. URL <https://arxiv.org/abs/2412.14964>.
- Dharshan Kumaran, Demis Hassabis, and James L McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in cognitive sciences*, 20(7): 512–534, 2016.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 3045–3059, 2021.
- Shengrui Li, Junzhe Chen, Xueting Han, and Jing Bai. Nuteprune: Efficient progressive pruning with numerous teachers for large language models, 2024. URL <https://arxiv.org/abs/2402.09773>.
- Zhiyuan Liang, Dongwen Tang, Yuhao Zhou, Xuanlei Zhao, Mingjia Shi, Wangbo Zhao, Zekai Li, Peihao Wang, Konstantin Schürholt, Damian Borth, Michael M. Bronstein, Yang You, Zhangyang Wang, and Kai Wang. Drag-and-drop llms: Zero-shot prompt-to-weights, 2025. URL <https://arxiv.org/abs/2506.16406>.
- Jiayu Liu, Zhenya Huang, Tong Xiao, Jing Sha, Jinze Wu, Qi Liu, Shijin Wang, and Enhong Chen. Socraticlm: Exploring socratic personalized teaching with large language models. *Advances in Neural Information Processing Systems*, 37:85693–85721, 2024.
- Dung Nguyen Manh, Thang Phan Chau, Nam Le Hai, Thong T. Doan, Nam V. Nguyen, Quang Pham, and Nghi D. Q. Bui. Codemmlu: A multi-task benchmark for assessing code understanding and reasoning capabilities of codellms, 2025. URL <https://arxiv.org/abs/2410.01999>.
- James L McClelland, Bruce L McNaughton, and Randall C O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological review*, 102(3):419–457, 1995.
- Martial Mermillod, Aurélie Bugaiska, and Patrick Bonin. The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects. *Frontiers in psychology*, 4:504, 2013.
- Xuefei Ning, Zifu Wang, Shiyao Li, Zinan Lin, Peiran Yao, Tianyu Fu, Matthew B. Blaschko, Guohao Dai, Huazhong Yang, and Yu Wang. Can llms learn by teaching for better reasoning? a preliminary study, 2024a. URL <https://arxiv.org/abs/2406.14629>.
- Xuefei Ning et al. Can llms learn from teaching? *arXiv preprint arXiv:2410.16451*, 2024b.
- Jean Piaget. *The origins of intelligence in children*. International Universities Press, 1952.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

- Rob Reich. The socratic method: What it is and how to use it in the classroom. *Speaking of Teaching*, 13(1):1–4, 2003.
- Henry L Roediger and Jeffrey D Karpicke. The power of testing memory: Basic research and implications for educational practice. *Perspectives on psychological science*, 1(3):181–210, 2006.
- Rod D Roscoe and Michelene TH Chi. Understanding tutor learning: Knowledge-building and knowledge-telling in peer tutors’ explanations and questions. *Review of educational research*, 77(4):534–574, 2007.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: an adversarial winograd schema challenge at scale. *Commun. ACM*, 64(9):99–106, August 2021. ISSN 0001-0782. doi: 10.1145/3474381. URL <https://doi.org/10.1145/3474381>.
- Maarten Sap, Hannah Rashkin, Derek Chen, Ronan LeBras, and Yejin Choi. Socialiqa: Commonsense reasoning about social interactions, 2019. URL <https://arxiv.org/abs/1904.09728>.
- Yao Shi, Rongkeng Liang, and Yong Xu. Educationq: Evaluating llms’ teaching capabilities through multi-agent dialogue framework. *arXiv preprint arXiv:2504.14928*, 2025.
- Avi Singh et al. Beyond human data: Scaling self-training for problem-solving with language models. *arXiv preprint arXiv:2312.06585*, 2024.
- Arjun Sinha et al. Maml-en-llm: Model-agnostic meta-learning for large language models. *arXiv preprint arXiv:2402.xxxxx*, 2024.
- John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive science*, 12(2): 257–285, 1988.
- Lev Semenovich Vygotsky. *Mind in society: The development of higher psychological processes*. Harvard university press, 1978.
- Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models. *arXiv preprint arXiv:2401.10491*, 2024a.
- Yuwei Wan, Yixuan Feng, Yanlin Yang, Yixin Li, and Jing Gao. Sciqag: A framework for auto-generated scientific question answering dataset with fine-grained evaluation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024b.
- Jiaxing Wang et al. Recurrent parameter generators. *arXiv preprint*, 2025.
- David Wood, Jerome S Bruner, and Gail Ross. The role of tutoring in problem solving. In *Journal of child psychology and psychiatry*, volume 17, pp. 89–100. Wiley, 1976.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xing He, Sainbayar Sukhbaatar, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence?, 2019. URL <https://arxiv.org/abs/1905.07830>.
- Liyi Zhang, Jake Snell, and Thomas L. Griffiths. Amortized bayesian meta-learning for low-rank adaptation of large language models, 2025a. URL <https://arxiv.org/abs/2508.14285>.
- Ming Zhang et al. Structtuning: Structure-aware training for large language models. In *International Conference on Learning Representations*, 2025b.
- Wen Zhang, Yang Feng, Fandong Meng, Di You, and Qun Liu. Bridging the gap between training and inference for neural machine translation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4334–4343, 2019.
- Yifan Zhang, Junqi Li, Yaguang Yang, Xiang Chen, Jing Gao, et al. Latentevolve: Self-evolving test-time scaling via latent optimization, 2025c.



Yue Zhang, Xin Chen, Lei Wang, et al. Yoda: Teacher-student progressive learning for language models. *arXiv preprint arXiv:2401.15670*, 2024a.

Zheyuan Zhang, Daniel Zhang-Li, Jifan Yu, Linlu Gong, Jinchang Zhou, Zhanxin Hao, Jianxiao Jiang, Jie Cao, Huiqin Liu, Zhiyuan Liu, Lei Hou, and Juanzi Li. Simulating classroom education with llm-empowered agents. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2024b.

Zheyuan Zhang, Daniel Zhang-Li, Jifan Yu, Linlu Gong, Jinchang Zhou, Zhanxin Hao, Jianxiao Jiang, Jie Cao, Huiqin Liu, Zhiyuan Liu, et al. Simulating classroom education with llm-empowered agents. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 10364–10379, 2025d.

## APPENDIX NAVIGATION INDEX

| Section                                         | Content                                                                                                                                                              | Pg Rf |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| <b>Theoretical Foundations</b>                  |                                                                                                                                                                      |       |
| A                                               | <b>Extended Cognitive Science Background</b> - Comprehensive theoretical grounding for LMT <sup>2</sup> , including Dreyfus, ACT-R, CLS, Bloom's, Piaget, & Vygotsky | 15    |
| <b>STRIDE Proof-of-Concept</b>                  |                                                                                                                                                                      |       |
| B                                               | <b>STRIDE Implementation Details</b> - Specifics for the STRIDE instantiation of LMT <sup>2</sup> , including architectural details and hyperparameter settings      | 21    |
| <b>LMT<sup>2</sup> Framework Implementation</b> |                                                                                                                                                                      |       |
| C                                               | <b>Phase I: The Student (Learning)</b> - Methodologies for Knowledge Gap Assessment (KGA), Teaching Material Curation and Interactive Concept Refinement             | 36    |
| D                                               | <b>Phase II: The Practitioner (Mastery)</b> - Details on Feedback-Driven Practice (Stage 5), Multi-Teacher Distillation (Stage 6) and Reflection (Stage 7)           | 48    |
| E                                               | <b>Phase III: The Teacher (Teaching)</b> - Implementation of Learning by Teaching (Stage 9) and student-to-teacher transition                                        | 58    |

## A EXTENDED COGNITIVE SCIENCE BACKGROUND

This appendix provides comprehensive theoretical grounding for the cognitive principles that motivate the LMT<sup>2</sup> framework. For each theory, we detail the psychological or neuroscientific evidence, explain why temporal separation of learning objectives is predicted to be beneficial and describe how the principle is operationalized in LMT<sup>2</sup> and validated through STRIDE.

## A.1 DEVELOPMENTAL AND EXPERTISE ACQUISITION THEORIES

## A.1.1 THE DREYFUS MODEL OF SKILL ACQUISITION

Stuart and Hubert Dreyfus Dreyfus & Dreyfus (1980) proposed a five-stage model of expertise development based on studies of chess players and airline pilots:

1. **Novice:** Follows explicit rules without context sensitivity
2. **Advanced Beginner:** Recognizes situational elements through experience
3. **Competent:** Plans and prioritizes among competing considerations
4. **Proficient:** Perceives situations holistically; sees what is salient
5. **Expert:** Responds intuitively; "knows what to do without reasoning"

The critical insight is that progression through these stages is *sequential and non-skippable*. Attempting to teach expert intuition to novices fails because they lack the foundational experiences that make intuition possible.

**LMT<sup>2</sup> Operationalization.** The nine-stage framework mirrors this progression: Stages 1-4 (Learning) correspond to novice/advanced beginner phases where explicit instruction dominates; Stages 5-8 (Mastery) correspond to competent/proficient phases where practice and feedback develop fluency; Stage 9 (Teaching) correspond to expert phases where articulating knowledge to others demonstrates and deepens mastery.

## A.1.2 FITTS-POSNER THREE-STAGE MODEL

Fitts and Posner Fitts & Posner (1967) identified three stages of motor and cognitive skill acquisition:

- **Cognitive Stage:** Learner understands what to do through explicit instruction; performance is slow, deliberate and error-prone

- **Associative Stage:** Practice refines performance; errors decrease; attention requirements reduce
- **Autonomous Stage:** Skill becomes automatic; can be performed with minimal conscious attention

**LMT<sup>2</sup> Operationalization.** Phase 1 (Learning) corresponds to the cognitive stage-the model receives explicit instruction and builds declarative representations. Phase 2 (Mastery) corresponds to the associative stage-practice and feedback refine performance. Phase 3 (Teaching) corresponds to autonomous mastery-the model can deploy knowledge flexibly and teach it to others.

### A.1.3 ACT-R THEORY OF SKILL COMPILATION

John Anderson’s ACT-R theory Anderson (1982; 2007) explains how declarative knowledge (facts, concepts) transforms into procedural knowledge (skills, procedures) through *knowledge compilation*:

- **Declarative Phase:** Knowledge exists as explicit facts that require conscious retrieval
- **Proceduralization:** Facts are compiled into condition-action production rules
- **Tuning:** Productions are refined through practice for speed and accuracy

The transition from declarative to procedural is not instantaneous-it requires extensive practice during which the learner “knows that” before they “know how.”

**LMT<sup>2</sup> Operationalization.** Stages 1-4 focus on declarative acquisition-the model learns *what* the domain knowledge is. Stages 5-8 implement proceduralization through practice-the model learns *how* to apply this knowledge. The temporal separation ensures declarative structures are stable before procedural compilation begins.

## A.2 LEARNING THEORY AND DEVELOPMENTAL PSYCHOLOGY

### A.2.1 BLOOM’S MASTERY LEARNING

Benjamin Bloom’s mastery learning framework Bloom (1968); Block (1971) revolutionized educational psychology by demonstrating that when instruction is properly structured, individual differences in learning outcomes can be dramatically reduced. The core principle: *learners must demonstrate mastery of prerequisite concepts before advancing to dependent ones.*

Bloom found that under mastery learning conditions, 80% of students achieved what only 20% achieved under traditional instruction. The key mechanisms include:

- **Formative assessment:** Frequent checks ensure prerequisites are met
- **Corrective instruction:** Remediates weaknesses before progression
- **Mastery criteria:** Clear standards prevent premature advancement

**LMT<sup>2</sup> Operationalization.** Stage completion gating implements mastery criteria-each stage must complete before the next begins. Stage 1 (Knowledge Gap Assessment) implements formative assessment. Stage 5 (Feedback-Driven Practice) implements corrective instruction. The critical insight is that Stage II (Knowledge-Task Alignment) should not begin until Stage I (Foundation Consolidation) is complete.

### A.2.2 PIAGET’S EQUILIBRATION THEORY

Jean Piaget’s equilibration theory Piaget (1952) describes cognitive development as progression through two complementary processes:

- **Assimilation:** Incorporating new information into existing mental structures (schemas)
- **Accommodation:** Modifying mental structures when existing schemas are inadequate

Stable learning requires these processes to reach *equilibrium*. The critical insight for AI training is that assimilation and accommodation impose *conflicting requirements* on knowledge representations:

- **Assimilation** requires representations to be *stable enough* to integrate new information coherently

- **Accommodation** requires representations to be *plastic enough* to modify when inadequate

When both are enforced simultaneously, the optimizer faces a stability-plasticity dilemma Mermillod et al. (2013): gradients that stabilize representations conflict with gradients that modify them.

**LMT<sup>2</sup> Operationalization.** Stage I (Foundation Consolidation) corresponds to assimilation-building stable domain structures. Stage II (Knowledge-Task Alignment) corresponds to accommodation-adapting structures to task requirements. Temporal separation allows each process to complete before the other begins, avoiding gradient interference.

### A.2.3 VYGOTSKY’S ZONE OF PROXIMAL DEVELOPMENT

Lev Vygotsky’s Zone of Proximal Development (ZPD) Vygotsky (1978) describes the space between what a learner can accomplish independently and what they can achieve with appropriate guidance:

- **Below ZPD:** Tasks are too easy; provide no learning signal
- **Within ZPD:** Tasks are challenging but achievable with support; maximal learning
- **Above ZPD:** Tasks are too difficult; learning signal is uninformative

**LMT<sup>2</sup> Operationalization.** Stage 2 (Teaching Material Curation) calibrates materials to the current model’s ZPD. Stage 5 (Feedback-Driven Practice) uses ZPD-calibrated question matrices that progress from easier to harder as competence develops. Critically, task-specific refinement (Stage III) is only effective *after* Stages I-II have expanded the model’s ZPD to include target tasks.

### A.2.4 COGNITIVE LOAD THEORY

John Sweller’s Cognitive Load Theory Sweller (1988) explains why simultaneous pursuit of multiple learning objectives degrades performance. Three types of cognitive load compete for limited working memory resources:

- **Intrinsic load:** Inherent complexity of the material
- **Extraneous load:** Load imposed by instructional design (e.g., conflicting objectives)
- **Germane load:** Load devoted to schema construction and learning

When extraneous load is high, germane load is necessarily reduced, degrading learning. The solution is to reduce extraneous load by presenting information sequentially rather than simultaneously.

**LMT<sup>2</sup> Operationalization.** Temporal separation eliminates the extraneous load of coordinating multiple objectives. In neural network terms, gradient interference between foundation consolidation ( $\mathcal{L}_1$ ) and task specialization ( $\mathcal{L}_2$ ) is eliminated by construction: when  $\langle \nabla \mathcal{L}_1, \nabla \mathcal{L}_2 \rangle < 0$ , the combined gradient  $\|\nabla \mathcal{L}_1 + \nabla \mathcal{L}_2\|^2 < \|\nabla \mathcal{L}_1\|^2 + \|\nabla \mathcal{L}_2\|^2$ , slowing progress on both objectives. Sequential staging avoids this entirely.

## A.3 NEUROSCIENCE AND MEMORY SYSTEMS

### A.3.1 COMPLEMENTARY LEARNING SYSTEMS THEORY

McClelland, McNaughton and O’Reilly’s Complementary Learning Systems (CLS) theory McClelland et al. (1995); Kumaran et al. (2016) explains how the brain avoids catastrophic forgetting through dual memory systems:

- **Hippocampus:** Fast learning of specific episodes; pattern separation
- **Neocortex:** Slow learning of generalizable knowledge; pattern completion

The hippocampus rapidly encodes specific experiences, which are then *replayed* during sleep to gradually train the neocortex. This temporal separation is critical: if the neocortex learned as fast as the hippocampus, new learning would overwrite old (catastrophic interference).

**LMT<sup>2</sup> Operationalization.** Stage 7 (Post-Instructional Reflection) implements dual-phase CLS through “Daytime” scaling (fast, episodic optimization) and “Nighttime” scaling (slow, procedural consolidation via the latent weaver). The episodic buffer stores specific high-quality experiences that are consolidated into generalizable knowledge.

### A.3.2 NEUROPLASTICITY AND STAGED LEARNING

Neuroplasticity-the brain’s ability to reorganize through experience Kolb & Whishaw (2009)-underlies all learning. However, plasticity is regulated: periods of high plasticity (critical periods) are followed by stabilization. This temporal structure ensures that foundational representations stabilize before dependent ones are built.

**LMT<sup>2</sup> Operationalization.** LoRA weight adaptation is computational neuroplasticity-experience-dependent changes to model parameters. The staged structure implements regulated plasticity: Stage I allows high plasticity for foundation consolidation, then “stabilizes” before Stage II introduces task-specific modifications. This preserves prior learning while enabling new adaptation.

## A.4 PEDAGOGY AND INSTRUCTIONAL DESIGN

### A.4.1 BRUNER’S SCAFFOLDING THEORY

Jerome Bruner’s scaffolding theory Wood et al. (1976), building on Vygotsky’s ZPD, describes how effective instruction provides temporary support that is gradually withdrawn as competence develops:

- **High scaffolding:** Novices receive extensive guidance, worked examples, explicit instruction
- **Faded scaffolding:** As competence grows, support is progressively removed
- **Independent performance:** Eventually learners perform without support

**LMT<sup>2</sup> Operationalization.** Stages 2-3 (Teaching Material Curation and Learning from Teacher) implement high scaffolding through adaptive teaching materials. As the model progresses, Stage 5 (Feedback-Driven Practice) provides progressively less guidance. By the Teaching phase (Stages 9-11), the model performs independently and teaches others.

### A.4.2 THE EXPERTISE REVERSAL EFFECT

Kalyuga et al. Kalyuga et al. (2003) demonstrated that instructional techniques optimal for novices can become counterproductive for experts:

- **Novices:** Benefit from worked examples, explicit guidance, high scaffolding
- **Experts:** Find same guidance redundant; prefer problem-solving, less structure

**LMT<sup>2</sup> Operationalization.** Stage 2 calibrates instructional approach to the model’s current capability. Materials generated for Stage I (when the model is a “novice”) differ from materials in Stage 5 (when the model has developed competence). This adaptive calibration prevents the expertise reversal effect.

### A.4.3 THE PROTÉGÉ EFFECT

The protégé effect Chase et al. (2009); Roscoe & Chi (2007); Biswas et al. (2005) demonstrates that teaching others produces deeper learning than passive review:

- **Knowledge reorganization:** Teaching requires explicit organization of implicit understanding
- **Gap identification:** Explaining exposes gaps in comprehension
- **Metacognitive monitoring:** Teachers must evaluate whether explanations succeed

**LMT<sup>2</sup> Operationalization.** Stage 9 (Learning by Teaching) implements the protégé effect for LLMs. The teacher model generates teaching materials for student models, receives feedback based on student performance (LbT scoring) and improves through DPO training on preference pairs. This produces stronger teachers through the weak-to-strong generalization phenomenon.

#### A.4.4 THE SOCRATIC METHOD

The Socratic method Reich (2003) uses questioning rather than direct instruction to promote deeper understanding. Questions force active engagement, expose assumptions and develop critical thinking.

**LMT<sup>2</sup> Operationalization.** Stage 4 (Interactive Concept Refinement) implements Socratic dialogue for doubt clearance. Stage 10 (Socratic-Style Mentoring) enables expert models to guide students through questioning rather than telling, developing student autonomy while deepening the mentor’s own understanding.

### A.5 METACOGNITION AND SKILL ACQUISITION

#### A.5.1 METACOGNITION AND SELF-REFLECTION

Flavell’s metacognition research Flavell (1979) demonstrates that “thinking about thinking”-monitoring and regulating one’s own cognitive processes-is a hallmark of expertise. Metacognitive skills include:

- **Planning:** Selecting strategies before problem-solving
- **Monitoring:** Tracking comprehension and progress during learning
- **Evaluating:** Assessing success and identifying improvement areas

**LMT<sup>2</sup> Operationalization.** Stage 7 (Post-Instructional Reflection) implements metacognition through self-rewarding test-time optimization. The model evaluates its own outputs, identifies areas for improvement and refines its reasoning process through latent-space optimization.

#### A.5.2 DELIBERATE PRACTICE

Anders Ericsson’s deliberate practice research Ericsson et al. (1993) identified that expertise develops not through simple repetition, but through structured practice with specific characteristics:

- **Clear goals:** Specific aspects of performance to improve
- **Immediate feedback:** Information about what’s working and what isn’t
- **Focused repetition:** Concentrated practice on weak areas
- **Progressive challenge:** Difficulty increases as competence develops

**LMT<sup>2</sup> Operationalization.** Stage 5 (Feedback-Driven Practice) implements deliberate practice. The YODA-inspired design provides clear goals (solve problems in weak topics), immediate feedback (teacher LLM evaluation and correction), focused repetition (weak topics receive more training samples) and progressive challenge (ZPD-calibrated difficulty progression).

#### A.5.3 THE TESTING EFFECT

Roediger and Karpicke Roediger & Karpicke (2006) demonstrated that retrieval practice (testing) enhances long-term retention more than passive review. The act of retrieving information strengthens memory traces.

**LMT<sup>2</sup> Operationalization.** Stage 5 implements the testing effect through MCQ-based practice. Stage 8 (Examination) provides formal assessment that both validates competence and strengthens retention. Questions are not merely evaluative-they are a training mechanism that enhances retention through retrieval.

### A.6 SUMMARY: CONVERGENT EVIDENCE FOR TEMPORAL SEPARATION

Sixteen theories from cognitive science, developmental psychology, neuroscience and educational research converge on the same prediction: *effective learning requires temporal separation of distinct objectives*. Whether described as phase transitions (Dreyfus, Fitts-Posner), memory system



complementarity (CLS), equilibration cycles (Piaget), or cognitive load management (Sweller), the fundamental principle remains consistent.

LMT<sup>2</sup> operationalizes this principle through an nine-stage framework organized into three phases-Learning, Mastery and Teaching-that mirrors the complete student-to-teacher lifecycle of human expertise development. STRIDE validates this principle through a minimal three-stage proof-of-concept demonstrating that cognitively-principled temporal separation consistently outperforms monolithic adaptation approaches.

## B STRIDE ADDITIONAL DETAILS

### B.1 HARDWARE AND COMPUTE.

All experiments were conducted on a single HPC node running Ubuntu 22.04.1. The system was equipped with an AMD EPYC 8434P CPU (48 physical cores, 96 logical threads), 256 GB of system RAM and four NVIDIA RTX A6000 GPUs, each with 48 GB of dedicated VRAM. GPU-accelerated workloads were executed using CUDA 12.4 and all experiments were implemented in Python 3.12.11.

The same hardware configuration was used for all methods, including our approach and all baselines, ensuring identical compute conditions and avoiding hardware-induced advantages. No system-level optimizations or specialized infrastructure beyond standard single-node multi-GPU execution were employed; reported performance differences therefore reflect algorithmic effects rather than differences in computational resources.

### B.2 STAGE 1 : FOUNDATIONAL KNOWLEDGE CONSOLIDATION MODULE

This section provides additional implementation details for the foundation consolidation stage (Stage 1) of STRIDE, focusing on (i) LoRA checkpoint collection and (ii) hyper-convolutional decoder architectures.

#### B.2.1 PROMPT ENCODING TENSOR STRUCTURE

In Eq. (4), prompt embeddings are constructed as

$$\begin{aligned} \mathbf{c}_i &= \text{Encoder}(p_i; \theta_{\text{enc}}), \\ \mathbf{C} &= [\mathbf{c}_1; \dots; \mathbf{c}_N] \in \mathbb{R}^{B \times N \times L \times C} \end{aligned} \quad (3)$$

Here:

- $B$  denotes the batch size (number of tasks processed in parallel),
- $N$  is the number of representative prompts sampled per task,
- $L$  is the token sequence length of each prompt,
- $C$  is the hidden embedding dimension of the frozen text encoder.

This tensorized representation follows the prompt-to-weights formulation introduced in Drag-and-Drop LLMs (DnD) Liang et al. (2025), and enables convolutional processing over task, prompt and token dimensions. Since  $B$  does not affect architectural design, it is omitted in subsequent architecture descriptions.

#### B.2.2 LORA CHECKPOINT COLLECTION PROTOCOL

STRIDE relies on a supervised prompt-to-weights mapping trained using LoRA checkpoints collected from task-specific fine-tuning runs. We adopt the checkpoint collection procedure proposed in DnD Liang et al. (2025).

Each checkpoint collection run consists of two phases:

- **Pretraining phase:** the base model is trained on the target dataset for a fixed number of steps using a higher learning rate.
- **Fine-tuning phase:** training continues with a lower learning rate, and a LoRA checkpoint is saved at each step.

Except for learning rate and training steps, all other hyperparameters (e.g., batch size, optimizer, data sampling strategy) are kept identical between the two phases. For datasets with fewer samples than the specified number, the full dataset is used.

Table 5 summarizes the checkpoint collection settings for different task families.

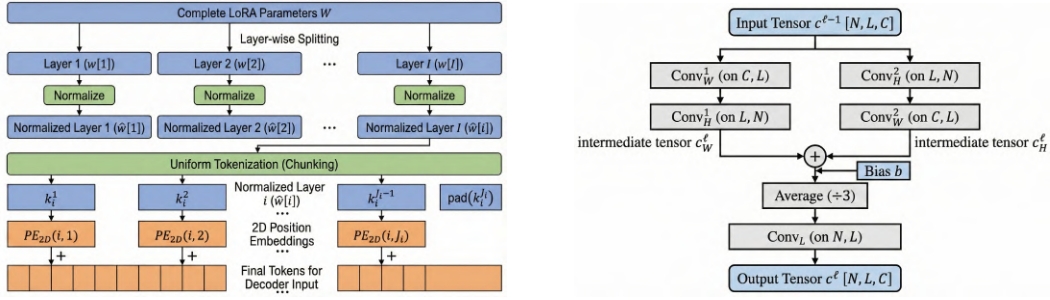


Figure 5: (a) Tokenization of LoRA parameters (*left*) and (b) Foundational Knowledge Consolidation Module Architecture (*right*)

| Task         | Pretraining        |       |            |           | Fine-Tuning        |       |
|--------------|--------------------|-------|------------|-----------|--------------------|-------|
|              | Learning Rate      | Steps | Batch Size | # Samples | Learning Rate      | Steps |
| Common Sense | $1 \times 10^{-4}$ | 75    | 32         | 5,000     | $1 \times 10^{-5}$ | 50    |

Table 5: Checkpoint collection settings for LoRA parameter generation

### B.2.3 PARAMETER TOKENIZATION

We employ a parameter tokenization strategy following Wang et al. (2025), which transforms LoRA adapter weights into a sequence of uniform tokens suitable for processing by the generalization model. It involves,

**Layer-wise splitting & normalization-** Given complete LoRA adapter parameters  $W$  spanning all layers, first parameters are segregated by layer index and then layer-wise normalization is applied to reduce distribution shifts across layers:

$$W \xrightarrow{\text{split by layer}} [w[1], \dots, w[I]] \xrightarrow{\text{normalize}} [\hat{w}[1], \dots, \hat{w}[I]] \quad (4)$$

**Uniform tokenization-** Each normalized layer  $\hat{w}[i]$  is then partitioned into contiguous, non-overlapping chunks of uniform size  $k$  (with padding applied to the final chunk if necessary):

$$\hat{w}[i] \xrightarrow{\text{tokenize}} K[i] = [k_i^1, k_i^2, \dots, \text{pad}(k_i^{J_i})], \quad (5)$$

where  $J_i$  denotes the number of tokens for layer  $i$  and  $\text{pad}(\cdot)$  indicates zero-padding to achieve uniform token length  $k$ . Each checkpoint  $W$  is then assigned a unique permutation state  $S$  encoded as a one-hot vector. Each token is further augmented with 2D sinusoidal position embeddings. For the  $j$ -th token in layer  $i$ ,  $e_i^j = \text{PE}_{2D}(i, j)$ , is computed, where the first dimension encodes layer index  $i$  and the second dimension encodes in-layer token position  $j$ .

For Qwen2.5-0.5B-Instruct with LoRA rank  $r = 8$ , each layer’s LoRA matrices have dimensions  $8 \times 896$ . With token size  $k = 1024$ , we obtain 7 tokens of size  $8 \times 128$  per layer, with the final token padded to  $10 \times 130$ . For Qwen2.5-1.5B-Instruct with  $r = 16$ , matrices of size  $16 \times 1536$  decompose into 6 tokens of  $16 \times 256$ , padded to  $18 \times 258$ . These tokenization schemes balance information density with computational tractability.

### B.2.4 HYPER-CONVOLUTIONAL DECODER ARCHITECTURES

The hyper-convolutional decoder maps prompt embedding tensors  $\mathbf{C} \in \mathbb{R}^{N \times L \times C}$  to tokenized LoRA parameters. We denote decoder architectures using a tuple  $(N, L, C)$  representing the input prompt dimension, token length and channel width respectively. Table 6 lists the decoder architectures used for different foundation model sizes.

| Size | Channel Progression ( $N, L, C$ )                                                                                                                                                                                    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.5B | $(n, 384, 384) \rightarrow (n, 200, 300) \rightarrow (n, 100, 256)$<br>$(2n, 50, 200) \rightarrow (4n, 50, 200) \rightarrow (8n, 25, 200)$<br>$(8n, 10, 200) \rightarrow (16n, 10, 200) \rightarrow (4296, 8, 128)$  |
| 1.5B | $(n, 384, 384) \rightarrow (n, 200, 300) \rightarrow (n, 100, 256)$<br>$(2n, 50, 200) \rightarrow (4n, 50, 200) \rightarrow (8n, 25, 200)$<br>$(8n, 10, 200) \rightarrow (16n, 10, 200) \rightarrow (4508, 18, 258)$ |

Table 6: Hyper-convolutional decoder architectures used for LoRA parameter generation across model sizes ( $n < 128$ )

### B.2.5 HYPERPARAMETER SUMMARY

This section summarizes hyperparameters and configuration choices that are explicitly specified in the main paper and appendix, collected here for ease of reproducibility.

**LoRA Configuration.** All experiments use LoRA adapters with identical parameterization across methods. For Qwen2.5-0.5B-Instruct, the LoRA rank is  $r = 8$ , yielding adapter matrices of size  $8 \times 896$ . For Qwen2.5-1.5B-Instruct, the rank is  $r = 16$ , yielding matrices of size  $16 \times 1536$ . LoRA adapters are applied to the query, key, value and output projections in attention layers, as well as the gate, up and down projections in MLP blocks. All base model parameters remain frozen.

**Foundation Consolidation (Stage 1).** LoRA checkpoints used for training the hyper-convolutional parameter generator are collected using a two-phase procedure. In the pretraining phase, models are trained for 75 steps with learning rate  $1 \times 10^{-4}$ . In the fine-tuning phase, training continues for 50 steps with learning rate  $1 \times 10^{-5}$ . Both phases use a batch size of 32 and up to 5,000 samples per dataset, as summarized in Table 5.

**Prompt Encoding and Decoder Architecture.** Task prompts are encoded using a frozen Sentence-BERT (all-MiniLM-L6-v2) encoder with maximum sequence length 384. The hyper-convolutional decoder consists of cascaded convolutional blocks, each containing five convolutional layers, with channel progressions specified in Table 6.

### B.2.6 DATASET DESCRIPTIONS

We evaluate STRIDE on a diverse collection of benchmarks spanning commonsense reasoning, mathematics, logic, social reasoning, medical question answering and code understanding. Following prior meta-learning and parameter-generation work Liang et al. (2025), we distinguish between *in-domain* tasks used during foundation consolidation and *out-of-domain* tasks used solely for evaluation.

**In-Domain Tasks.** By in-domain tasks, we refer to tasks that are included during the foundation consolidation stage (Stage 1), following a leave-one-out meta-training protocol. ARC-Challenge and ARC-Easy Clark et al. (2018) contain grade-school-level multiple-choice science questions designed to test elementary reasoning. HellaSwag Zellers et al. (2019) evaluates commonsense inference by requiring models to select the most plausible continuation of a given context from adversarially constructed alternatives. BoolQ Clark et al. (2019) consists of naturally occurring yes/no questions derived from real-world passages. PIQA Bisk et al. (2019) focuses on physical commonsense reasoning in everyday situations, requiring selection of the most plausible solution. WinoGrande Sakaguchi et al. (2021) is a large-scale dataset for commonsense reasoning framed as a fill-in-the-blank task with binary choices.

**Out-of-Domain Tasks.** Out-of-domain tasks are never used during foundation consolidation and serve to evaluate robustness under domain shift. GSM-8K Cobbe et al. (2021) consists of grade-school mathematical word problems requiring multi-step arithmetic reasoning. MATH Hendrycks et al. (2021) contains challenging competition-level mathematics problems spanning algebra, geometry and number theory. DivLogicEval Chung et al. (2025) assesses logical reasoning through counterintuitive natural-language questions designed to isolate pure logical inference. SocialIQA Sap et al. (2019)

evaluates social and emotional commonsense reasoning in everyday interactions. CodeMMLU Manh et al. (2025) is a multitask benchmark for code understanding, covering program analysis, bug detection and software engineering concepts across multiple programming languages. The JAMA Clinical Challenge Chen et al. (2025a) consists of expert-curated medical case questions with detailed explanations, designed to evaluate clinical reasoning and decision-making.

### B.3 STAGE 2: KNOWLEDGE-TASK ALIGNMENT

This section provides detailed implementation information for the Fusion VAE used in Stage 2 (Knowledge-Task Alignment).

#### B.3.1 TRAINING DATA PREPARATION

The LoRA checkpoints collected during Stage 1 for training the parameter generator are reused for Fusion VAE training. For each checkpoint, we compute its corresponding task vector using the training dataset of that task. Specifically:

- LoRA parameters are flattened into 1D vectors for processing
- Task vectors are computed as the difference between fine-tuned and pre-trained model outputs on task-specific prompts
- Training pairs  $(l^{(i)}, v_{\mathcal{T}_i})$  are constructed from LoRA parameters and corresponding task vectors

#### B.3.2 FUSION VAE ARCHITECTURE

The VAE employs a 1D convolutional encoder-decoder architecture:

##### Encoder.

- 5-layer 1D CNN with channel progression  $[1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512]$
- Kernel size: 4, Stride: 4
- Adaptive average pooling to spatial dimension 32
- Separate linear heads for mean  $\mu$  and log-variance  $\log \sigma^2$

**Task Vector Integration.** Task vectors are projected to 512 dimensions via a linear layer and concatenated with compressed LoRA features before latent encoding. The decoder receives both the sampled latent  $\mathbf{z}$  and the projected task vector for reconstruction.

**Decoder.** Mirror architecture of the encoder using transposed convolutions, with final output matching the original LoRA parameter dimension.

#### B.3.3 TRAINING PROCEDURE

Training follows a meta-learning framework with the following hyperparameters:

- **Meta-epochs:** 4000
- **Inner loop steps:** 1
- **Inner learning rate:**  $1 \times 10^{-3}$
- **Meta learning rate:**  $1 \times 10^{-4}$
- **KL weight:** Annealed from 0 to 0.005 over training
- **Optimizer:** Adam with default parameters

The loss function combines reconstruction error (MSE) and KL divergence regularization:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + \lambda_{\text{KL}} \cdot \mathcal{L}_{\text{KL}} \quad (6)$$

**Algorithm 1** Stage II: Task-Conditioned Fusion Alignment via Conditional VAE

---

**Require:** Training tasks  $\{\mathcal{T}_i\}$ , generator outputs  $\theta_{\text{gen}}^a$ , encoder/decoder parameters  $(\phi, \psi)$ , KL weight  $\lambda_{\text{KL}}$

- 1: **for** each alignment iteration **do**
- 2:   Sample task  $\mathcal{T}_i \sim p(\mathcal{T})$
- 3:   Compute task semantic signature  $\mathbf{v}_{\mathcal{T}_i}$
- 4:   Encode conditional posterior:  $(\mu_i, \sigma_i) \leftarrow q_\phi(\mathbf{z} \mid \theta_{\text{gen}}^a, \mathbf{v}_{\mathcal{T}_i})$
- 5:   Sample latent:  $\mathbf{z}_i \sim \mathcal{N}(\mu_i, \text{diag}(\sigma_i^2))$
- 6:   Decode aligned adapter:  $\theta_{\text{aligned}}^a \leftarrow \text{Dec}_\psi(\mathbf{z}_i, \mathbf{v}_{\mathcal{T}_i})$
- 7:   Reconstruction loss:  $\mathcal{L}_{\text{recon}} \leftarrow \left\| \theta_{\text{gen}}^a - \theta_{\text{aligned}}^a \right\|^2$
- 8:   KL regularization:  $\mathcal{L}_{\text{KL}} \leftarrow D_{\text{KL}}(q_\phi(\mathbf{z} \mid \cdot) \parallel \mathcal{N}(0, I))$
- 9:   Full alignment objective:  $\mathcal{L}_{\text{align}} \leftarrow \mathcal{L}_{\text{recon}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}}$
- 10:   Update encoder/decoder:  $(\phi, \psi) \leftarrow (\phi, \psi) - \eta \nabla_{\phi, \psi} \mathcal{L}_{\text{align}}$
- 11: **end for**
- 12: **Return:** aligned adapter distribution  $\theta_{\text{aligned}}^a$  for downstream refinement.

---

**B.3.4** ALGORITHMIC FORMULATION OF STAGE II ALIGNMENT

For completeness, we provide the explicit optimization procedure used to train the task-aware conditional VAE in Stage II. The objective is to transform the generated adapter parameters from Stage I into semantically aligned parameters that respect the target task geometry before refinement.

**B.3.5** INFERENCE PIPELINE

At inference time for an unseen task:

1. **Initial LoRA Generation:** Generate initial LoRA parameters using the Stage 1 parameter generator from target task prompts
2. **Task Vector Computation:** Compute task vector from target task prompts using the pre-trained model
3. **VAE Encoding:** Pass generated LoRA parameters and task vector through the Fusion VAE encoder to obtain latent representation
4. **Aligned Decoding:** Sample from the latent distribution and decode conditioned on the task vector to obtain aligned LoRA parameters

This process ensures that the output LoRA parameters are semantically aligned with the target task while preserving the foundational knowledge consolidated in Stage 1.

**B.4** STAGE 3: TASK-SPECIFIC REFINEMENT

This section provides a comprehensive treatment of the four adaptation strategy families available in Stage III of STRIDE: Test-Time Learning (TTL), Two-Subspace Mixing LoRA, Test-Time Scaling (TTS) and Latent Space Modification. Each strategy represents a bounded inference-time transformation that can be selected by the reinforcement learning-based refinement policy  $\pi_\phi$  to optimize task-specific performance.

Recall from Section 4 that refinement is cast as a decision-making problem:

$$\alpha^* = \arg \max_{\alpha \in \mathcal{A}} \mathbb{E}_{\alpha \sim \pi_\phi(\cdot \mid \tau)} [R(\tau, \alpha)], \quad (7)$$

where each  $\alpha \in \mathcal{A} = \{\text{TTL}, \text{LoRA}, \text{TTS}, \text{Latent}, \dots\}$  specifies a concrete adaptation operator with an explicit hyperparameter configuration. The strategy model emits structured JSON outputs that are parsed deterministically at deployment.

**Algorithm 2** TTL: Test-Time Learning via Weighted Perplexity Minimization

---

**Require:** Test batch  $\mathcal{X} = \{x_b\}_{b=1}^B$ , pretrained LLM  $f_\Theta$ , LoRA params  $\Delta\Theta$ , threshold  $\mathcal{P}_0$

- 1: Initialize LoRA:  $\mathbf{A} \sim \mathcal{N}(0, \sigma^2)$ ,  $\mathcal{B} = 0$
- 2:  $\tilde{\Theta} \leftarrow \Theta + \Delta\Theta$
- 3: **for** each batch  $\mathcal{X}$  **do**
- 4:   Compute perplexities  $\mathcal{P}(x_b; \tilde{\Theta})$
- 5:   Compute weights  $S(x_b)$ ; exclude low-perplexity samples
- 6:   Update  $\Delta\Theta$  by minimizing  $\sum_{b=1}^B S(x_b) \mathcal{P}(x_b; \tilde{\Theta})$
- 7: **end for**
- 8: **Return:** adapted model  $f_{\Theta+\Delta\Theta}$

---

**B.4.1 TEST-TIME LEARNING (TTL)**

Test-Time Learning adapts the model using only unlabeled test inputs by minimizing input perplexity. The key insight is that reducing perplexity on a question  $x$  implicitly improves answer quality when the question-answer pair is semantically aligned.

**Problem Setting.** Let  $f_\Theta$  be a pretrained autoregressive LLM with frozen base parameters  $\Theta$ . Given unlabeled test inputs  $\mathcal{D}_{\text{Test}} = \{x_j\}_{j=1}^M$ , TTL adapts the model by updating only lightweight LoRA parameters  $\Delta\Theta$ , yielding adapted parameters  $\tilde{\Theta} = \Theta + \Delta\Theta$ .

**Perplexity Objective.** For token sequence  $x = (x_1, \dots, x_T)$ , perplexity is defined as:

$$\mathcal{P}(x; \Theta) = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p(x_t \mid x_{1:t-1}; \Theta)\right). \quad (8)$$

Since ground-truth answers  $y$  are unavailable at test time, TTL minimizes *input* perplexity:  $\min_{\Theta} \mathcal{P}(x; \Theta)$ .

**Sample-Efficient Weighting.** Not all test samples contribute equally. TTL uses a perplexity-based selection weight that prioritizes high-perplexity samples:

$$S(x) = \lambda \cdot \exp(\log \mathcal{P}(x; \Theta) - \log \mathcal{P}_0) \mathbb{I}_{\{\mathcal{P}(x; \Theta) > \mathcal{P}_0\}}(x), \quad (9)$$

where  $\mathcal{P}_0$  is a threshold and  $\lambda$  is a scaling constant. Low-perplexity samples are excluded ( $S(x) = 0$ ), focusing adaptation on samples where the model is most uncertain.

**LoRA-Based Adaptation.** To prevent catastrophic forgetting and reduce compute, TTL updates only low-rank parameters  $\Delta\Theta = \mathcal{B}\mathbf{A}$ , yielding the weighted objective:

$$\min_{\Delta\Theta} S(x) \mathcal{P}(x; \Theta + \Delta\Theta). \quad (10)$$

**Configuration Schema.**

```
{
  "family": "TTT",
  "ttl_steps": <integer>,
  // number of optimization steps
  "learning_rate": <float>,
  // optimizer learning rate
  "batch_size": <integer>,
  // samples per update
  "shuffle_data": <boolean>
  // whether to shuffle test inputs
}
```



#### B.4.2 TWO-SUBSPACE MIXING LORA

This strategy enhances standard LoRA by enabling subspace interaction through a fixed butterfly mixing factor, providing richer representational capacity without additional trainable parameters.

**LoRA as Subspace Composition.** For frozen pretrained weights  $\mathbf{W}_0 \in \mathbb{R}^{d_1 \times d_2}$  and input  $x$ , standard LoRA computes:

$$x\mathbf{W}_0 + x\Delta\mathbf{W} = x\mathbf{W}_0 + x\mathbf{A}\mathbf{B}, \quad (11)$$

with  $\mathbf{A} \in \mathbb{R}^{d_1 \times r}$ ,  $\mathbf{B} \in \mathbb{R}^{r \times d_2}$  and  $r \ll \min(d_1, d_2)$ . Decomposing into rank-1 components:

$$\mathbf{A} = [\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_r], \quad \mathbf{B}^\top = [\mathbf{B}_1^\top, \mathbf{B}_2^\top, \dots, \mathbf{B}_r^\top],$$

vanilla LoRA can be viewed as:

$$x\mathbf{A}\mathbf{B} = x \sum_{i=1}^r \mathbf{A}_i \mathbf{B}_i = x\mathbf{A}\mathbf{I}_{r \times r}\mathbf{B}. \quad (12)$$

**Two-Subspace Mixing.** The two-subspace mixing variant replaces the identity mixer with a butterfly factor that enables cross-subspace interaction:

$$x \sum_{i=1}^{r/2} (\mathbf{A}_i + \mathbf{A}_{i+r/2})(\mathbf{B}_i + \mathbf{B}_{i+r/2}) = x\mathbf{A} \begin{bmatrix} \mathbf{I} & \mathbf{I} \\ \mathbf{I} & \mathbf{I} \end{bmatrix} \mathbf{B}. \quad (13)$$

This formulation mixes information from  $2r$  subspaces (compared to  $r$  in vanilla LoRA), modeling richer interactions without introducing additional learnable weights.

**Interpolation Parameter.** The  $\lambda$  hyperparameter controls the interpolation ratio between the two resulting subspace outputs during inference, allowing fine-grained control over the adaptation strength.

#### Configuration Schema.

```
{
  "family": "LORA",
  "lambda": <float>
  // mixing ratio between subspaces
  // (0.0 to 1.0)
}
```

#### B.4.3 TEST-TIME SCALING (TTS)

Test-Time Scaling leverages multiple prompt batches to improve prediction stability through either routing or ensembling. This approach is particularly effective for tasks where multiple plausible interpretations compete.

**Router Approach.** Given a test question, TTS samples  $m$  prompt batches from the test split and selects the batch whose representation is closest to the question. Two routing methods are supported:

- **M1 (avg\_sim\_score):** Compute similarity scores between each prompt in a batch and the test question, then average across the batch. Select the batch with highest average similarity.
- **M2 (avg\_prompt\_embed):** Compute the mean prompt embedding for each batch, then select the batch whose mean embedding is closest to the test question embedding.

Euclidean distance was found to perform better empirically than cosine similarity for both methods.

**Ensemble Approach.** For tasks requiring aggregation across multiple adapter configurations:

- **max\_confidence:** Select the prediction with highest confidence score across all configurations.
- **majority\_vote:** Aggregate predictions via voting across configurations.
- **sum\_logprobs:** Sum log-probabilities across configurations for each candidate answer.

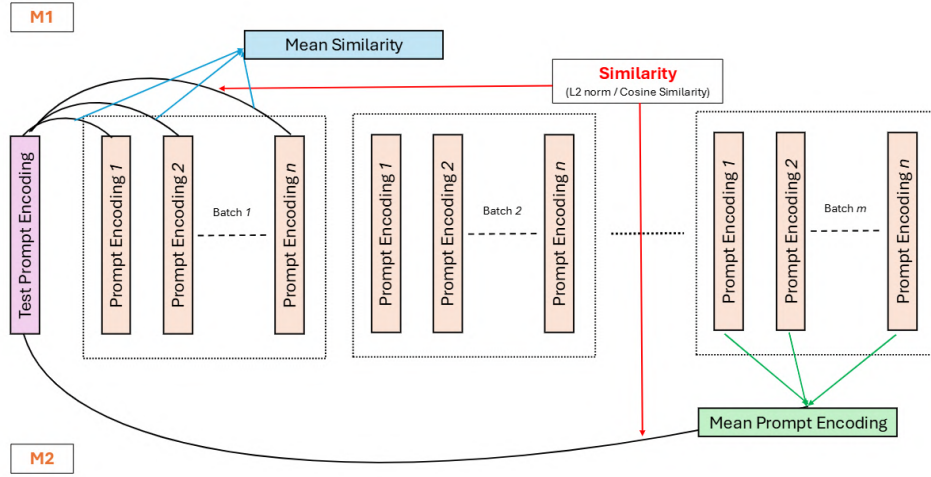


Figure 6: **TTS Router Architecture.** M1 computes mean similarity scores between the test prompt encoding and individual prompt encodings within each batch. M2 computes similarity between the test encoding and mean prompt embeddings per batch.

#### Configuration Schema.

```
{
  "family": "TTS",
  "num_prompt_batches": <integer>,
  // batches sampled from test split
  "method": "<avg_sim_score |
    avg_prompt_embed |
    max_confidence |
    majority_vote |
    sum_logprobs>"
}
```

#### B.4.4 LATENT SPACE MODIFICATION

We introduce a lightweight, per-sample parameter applied to the final hidden layer, enabling rapid adaptation without modifying backbone parameters.

**Problem Setting.** Let  $\mathcal{M}$  be a pretrained autoregressive LM with fixed parameters  $\theta$ . Given input prompt  $x = (x_1, \dots, x_n)$ , the model produces final hidden features  $H = \mathcal{M}_{\text{pre-LM}}(x) \in \mathbb{R}^{n \times d}$  before the LM head. Next-token probabilities are  $p(y | x) = \text{softmax}(W_{\text{LM}}H)$ .

**Sample-Specific Parameter.** SLOT introduces a per-sample additive parameter  $\delta \in \mathbb{R}^{1 \times d}$ , broadcast across sequence positions:

$$H' = H + \delta, \quad \text{logits} = W_{\text{LM}}(H + \delta). \quad (14)$$

**Prompt-Stage Optimization.** Initialize  $\delta^{(0)} = \mathbf{0}$  and optimize for  $T$  steps by minimizing the negative log-likelihood of the prompt sequence:

$$\mathcal{L}(\delta) = - \sum_{i=1}^{n-1} \log p(x_{i+1} | x_{1:i}, \delta). \quad (15)$$

Since  $\delta$  is applied only to the final hidden layer, features  $H$  can be cached; each optimization step requires only forward/backward through the linear head  $W_{\text{LM}}$ , yielding negligible overhead.

**Algorithm 3** SLOT: Sample-Specific LM Optimization at Test-Time

---

```

1: Input: pretrained LM  $\mathcal{M}(\theta)$ , prompt  $x$ , steps  $T$ , lr  $\eta$ 
2: Initialize  $\delta \leftarrow 0 \in \mathbb{R}^{1 \times d}$ 
3: for  $t = 0$  to  $T - 1$  do
4:    $H \leftarrow \mathcal{M}_{\text{pre-LM}}(x)$  (can be cached)
5:    $\mathcal{L}(\delta) \leftarrow -\sum_{i=1}^{n-1} \log p(x_{i+1} \mid x_{1:i}, \delta)$ 
6:    $\delta \leftarrow \text{OptimizerStep}(\delta, \nabla_{\delta} \mathcal{L}; \eta)$ 
7: end for
8: Decode: generate with logits  $W_{\text{LM}}(H_{\text{last}} + \delta)$ 
9: Return: generated continuation  $y$ 

```

---

**Generation Stage.** During autoregressive decoding, SLOT reuses  $\delta_{\text{opt}}$  without further optimization:

$$x_{\text{next}} \sim \text{softmax}(W_{\text{LM}}(H_{\text{last}} + \delta_{\text{opt}})). \quad (16)$$

**Computational Cost.** Prompt-stage adaptation optimizes only  $d$  parameters with per-step cost  $O(d|V|)$  and generation-time overhead  $O(d)$  per token.

**Configuration Schema.**

```

{
  "family": "Latent",
  "slot_steps": <integer>,
  // optimization steps (T)
  "slot_lr": <float>
  // learning rate (eta)
}

```

**B.4.5 STRATEGY SELECTION VIA REINFORCEMENT LEARNING**

The refinement policy  $\pi_{\phi}$  selects strategies through a language-model-driven generation procedure. Given task context and few-shot exemplars, the strategy model emits structured JSON outputs specifying both the selected family and its configuration. This ensures refinement policies remain interpretable and directly executable.

**Objective Formulation.** We first formulate the objective for outer-loop RL training which generates adaptation strategies  $\alpha$ . Let  $\theta$  denote the parameters of the language model  $\text{LM}_{\theta}$ . In order to adapt to an unseen dataset (task)  $\mathcal{D}$ , STRIDE requires as specified in Section 4,  $\tau$  which is a context containing information relevant to the task and  $\mathcal{E}$  which is the evaluation strategy and metric used to assess the model’s downstream adaptation. Based on  $\tau$ , STRIDE generates an  $\alpha$  and updates its parameters accordingly  $\theta' \leftarrow \text{Update}(\theta, \alpha)$ . We thus have an RL setup i.e., the model takes an *action* (generating  $\alpha$ ), receives a *reward*  $r$  based on  $\text{LM}_{\theta'}$ ’s performance on  $\mathcal{E}$  and updates its policy to maximize expected reward,

$$\mathcal{L}_{\text{RL}}(\theta_t) := -\mathbb{E}_{(\tau, \mathcal{E}) \sim \mathcal{D}} \left[ \mathbb{E}_{\alpha \sim \text{LM}_{\theta_t}(\cdot | \tau)} [r(\alpha, \mathcal{E}, \theta_t)] \right]$$

It is to be noted that the reward assigned to a given action depends on the model parameters  $\theta$  at the time the action is taken (since  $\theta$  is updated to  $\theta'$ , which is then evaluated). An implication of this is that the while modeling the RL state, one must therefore include  $\theta$  in the policy’s parameters as well along with  $\tau$ , even though the policy’s observation is limited to  $\tau$  (because it is extremely infeasible to directly place  $\theta$  in the LLM’s context window). Therefore, the (state, action, reward) triples which have been collected by using an older model weights,  $\theta_{\text{old}}$ , will not be aligned for the current model  $\theta_{\text{current}}$ . Hence, an on-policy approach should be adapted, by which adaptation strategies are sampled from and, even more importantly, the rewards itself will be calculated using the current model.

In particular, the specific on-policy approach used is  $\text{ReST}^{\text{EM}}$  Singh et al. (2024) where samples are first generated from the current model and are filtered by using binary feedback  $[r(\alpha, \mathcal{E}, \theta_t)]$  is 1 if

on  $\mathcal{E}$ ,  $\alpha$  improves  $\text{LM}_{\theta_t}$ 's performance and is 0 otherwise]. The model is then fine-tuned on these samples and this continues in an iterative manner. Currently, only a deterministic number of samples are being generated, 20 to be precise. This could however be improvised to be dynamic in future version of the work wherein samples would continue to be generated until a particular confidence threshold, as determined by the model itself is reached instead. The same is true for number of iterations as well which is just 2 for now. The generation process employs a temperature of 1.0, nucleus sampling ( $\text{top-p} = 0.9$ ) and top-k filtering ( $\text{top-k} = 50$ ).

**Dataset-Specific Configurations.** Empirically discovered configurations exhibit intuitive alignment with task demands:

- **ARC-e, JAMA Clinical, PIQA  $\rightarrow$  TTL:** Domain-specific or jargon-dense tasks benefit from token-level distribution correction via perplexity minimization. Example Configuration: `{ttl_steps: 25, learning_rate: 1e-5, batch_size: 4}`.
- **ARC-c, SocialIQa  $\rightarrow$  Latent:** Abstract reasoning and social inference tasks require adjustment of hidden-state trajectories. Example Configuration: `{slot_steps: 5, slot_lr: 0.1}`.
- **BoolQ, GSM-MC, MATH-MC  $\rightarrow$  LoRA:** Structured reasoning tasks benefit from subspace mixing. Example Configuration: `{lambda: 0.5}`.
- **HellaSwag, DivLogicEval, CodeMMLU  $\rightarrow$  TTS:** Adversarial or multi-interpretation tasks benefit from ensemble-style aggregation. Example Configuration: `{num_prompt_batches: 20, method: max_confidence}`.

#### B.4.6 JSON SCHEMAS AND PROMPTING TEMPLATE FOR STRATEGY GENERATION

For reproducibility, we provide the exact prompting template used to generate adaptation strategies. The model is instructed to output a single valid JSON object corresponding to one strategy family, with no additional text. Task context's are obtained using by prompting a foundational model with the few-shot examples of the unseen task analogous to Charakorn et al. (2025).

##### System Prompt.

```
You are an expert AI agent tasked with generating an optimal
adaptation strategy for a Large Language Model to improve its
performance on a new, unseen task. Your goal is to output a single,
structured JSON object that specifies the most promising strategy.
```

##### User Prompt.

```
<Task Context>

Your Instruction:
Based on the task context and method descriptions below, generate a
single JSON
object representing the most effective adaptation strategy. You must
choose one
strategy family and output strictly valid JSON with no extra text.

JSON Output Format \& Strategy Families:

(1) Test-Time Training (TTT)
{
  "family": "TTT",
  "ttl_steps": <integer>,
  "learning_rate": <float>,
  "batch_size": <integer>,
  "shuffle_data": <boolean>
}
```

```

(2) LoRA Modification (LORA)
{
  "family": "LORA",
  "lambda": <float>
}

(3) Test-Time Scaling (TTS)
{
  "family": "TTS",
  "num_prompt_batches": <integer>,
  "method": "<avg_sim_score | avg_prompt_embed | max_confidence |
majority_vote | sum_logprobs>"
}

(4) Latent Space Modification (Latent)
{
  "family": "Latent",
  "slot_steps": <integer>,
  "slot_lr": <float>
}

Now, provide only the JSON object for the <task> dataset.

```

### Task Contexts

*Generation Prompt:* This prompt is used for querying GPT-4o mini to obtain the task descriptions for datasets

You are given a small set of example question-answer pairs from an unknown dataset. Your task is to infer the underlying task definition and write a concise, professional task description suitable for inclusion in a machine learning benchmark paper.

Instructions:

1. Do not mention specific example questions or answers.
2. Infer the core objective, skills being evaluated and type of reasoning required.
3. Describe what the model is expected to do and what competencies are being tested.
4. Write in neutral, academic language.

Output a single self-contained task description paragraph.

Examples from the dataset are provided below:

<question-answer examples>

Output only the task description. Do not include analysis, bullet points, or headings.

Hereby, are the example task contexts used in this work, **(a) ARC-c**

This task is about analyzing questions which examine your grasp of scientific ideas. You must connect conceptual knowledge with practical examples from geology, ecology and environmental changes. The objective here is to evaluate various scientific scenarios and infer the most logical explanations or definitions based on established knowledge. This task will strengthen your analytical and reasoning skills in the context of natural science. Your role is to interpret questions focusing on earth science and biological interactions. This demands a clear understanding of relevant processes, such as decomposition, weathering and species adaptation.

**(b) ARC-e**

Your job is to discern which information best answers a posed question, focusing on practical examples and scientific principles. This requires a strong grasp of underlying concepts in ecology or physics. You will analyze questions that explore important connections such as environmental issues or animal adaptations. Utilize your background knowledge to evaluate and select the most fitting answer. This task involves selecting answers that reflect accurate relationships or effects seen in nature or society. You will need to sort through potential choices critically to find the appropriate one.

### (c) HellaSwag

This task revolves around completing an unfinished text by selecting an ending that matches its tone and context. It requires you to think critically about how narratives develop and conclude effectively. This task asks you to select a suitable conclusion for an unfinished narrative or instructional content. It tests your comprehension and reasoning skills as you assess how well each option aligns with the given text. Your task involves completing an incomplete passage by selecting the ending that logically continues the context provided. This requires reading comprehension and the ability to infer meaning from a text.

### (d) PIQA

You will explore practical questions and select an answer that presents a logical and widely accepted approach to solve a given problem or complete a task successfully. Analyze the provided scenarios where practical advice or solutions are required, focusing on selecting the most commonly used or convenient method. Given a question related to common tasks, your responsibility is to discern which proposed solution aligns with typical practices or makes the task easier to achieve.

### (e) WinoGrande

In this exercise, you need to read short narratives and discern which person or object fits best within the context of the sentence. This task requires synthesizing information from concise textual scenarios to identify crucial elements that drive the narrative forward. The goal is to evaluate descriptions and select the entity that best aligns with the sentiments or actions presented in the scenario.

### (f) BoolQ

Analyze the given details about various subjects, including movies, sports and television shows. Your role is to confirm whether certain claims are true or false. Your task is to determine the truthfulness of specific statements based on the provided background information. This requires careful reading and comprehension of the content. The goal is to evaluate factual claims made in relation to highlighted texts. You will need to discern whether the statements align with the information provided.

### (g) GSM-8K

You will be tasked with interpreting mathematical situations described in words. The goal is to use logical

reasoning and calculations to determine the numerical answers based on the context provided. This task challenges your problem-solving abilities through mathematical reasoning. You must carefully read each scenario and systematically work through the data to compute the final outcome. Your role is to engage with practical math scenarios presented as questions. The task requires translating textual data into numerical operations that will lead you to the final solution.

#### (h) MATH

This task focuses on solving challenging mathematical problems that require multi-step logical reasoning rather than direct formula application. You will analyze competition-level mathematics questions spanning topics such as algebra, geometry, number theory, probability and calculus. The objective is to carefully interpret each problem, identify appropriate problem-solving strategies and carry out precise symbolic or numerical reasoning to arrive at a correct final answer. This task emphasizes structured thinking, the use of mathematical heuristics and the ability to connect multiple concepts within a single solution. Your role is to reason through complex scenarios, perform intermediate derivations when necessary and produce an exact answer that adheres to standard mathematical conventions. The task evaluates deep mathematical understanding and disciplined reasoning rather than surface-level computation or pattern matching.

#### (i) DivLogicEval

This task focuses on evaluating your ability to perform precise logical reasoning over natural language statements. You will be given a set of premises written in fluent but often counterintuitive language, where commonsense intuition alone may be misleading. Your objective is to analyze the logical structure underlying these statements and determine which option is logically entailed, not entailed, or required as a missing assumption. The task demands careful attention to implications, negations and dependencies between statements rather than surface-level meaning. You must rely on formal reasoning principles to assess whether conclusions follow from the premises. This task is designed to isolate logical reasoning skills by minimizing reliance on background knowledge or real-world plausibility.

#### (j) SocialIQA

This task focuses on reasoning about everyday social situations that involve human interactions, intentions and emotional responses. You are given a short context describing a social scenario, followed by a question that probes implicit social commonsense. Your objective is to select the most plausible answer from multiple choices based on how people typically think, feel, or act in such situations. The questions require you to infer motivations, emotional reactions, social norms, or likely actions before or after an event. This task evaluates your ability to reason about social dynamics, perspective-taking and cause-effect relationships in human behavior. Successfully completing this task demands an understanding of common social expectations and the ability to apply Theory of Mind reasoning to interpret the mental states of individuals involved.

#### (k) CodeMMLU

This task focuses on evaluating your understanding of programming concepts, code semantics and software reasoning across a wide range of difficulty levels. You will analyze questions that involve reading, interpreting and reasoning about code snippets written in common programming languages. The objective is to assess your ability to understand control flow, data structures, algorithms and language-specific behaviors. This task requires careful examination of program logic, identification of errors or expected outputs and selection of the most appropriate answer based on correct computational reasoning. Your role is to apply foundational and advanced coding knowledge to infer how programs execute and how modifications affect their behavior. The task emphasizes precise reasoning about syntax, semantics and program execution rather than surface-level pattern matching.

### (I) JAMA Clinical

This task focuses on answering and explaining complex real-world clinical cases drawn from challenging medical scenarios. You are required to analyze detailed patient case descriptions that may include atypical presentations, incomplete information, or competing diagnoses. The objective is to apply clinical reasoning to identify the most appropriate diagnosis or next management step from multiple answer choices. Beyond selecting the correct option, this task emphasizes understanding why that choice is correct and why alternative options are less appropriate. Successfully completing this task requires synthesizing medical knowledge across domains, interpreting clinical findings and reasoning in a manner consistent with expert decision-making in real clinical settings. The task evaluates both diagnostic accuracy and the ability to justify medical decisions using coherent, medically sound explanations.

## B.5 VALIDATING TEMPORAL SEPARATION PRINCIPLE

Before presenting quantitative performance results, we first analyze the behavioral differences between STRIDE and bi-level baselines to validate our core hypothesis: **temporal separation eliminates gradient interference and enables genuine task-specific adaptation**. The analyses in this section demonstrate that STRIDE’s performance gains arise from structurally resolving the coupled optimization conflict, not from incidental hyperparameter choices or additional capacity.

### B.5.1 GRADIENT CONFLICT ANALYSIS

We analyze gradient structure across out-of-domain tasks by computing pairwise cosine similarity between task-specific gradient vectors. Higher similarity with lower variance indicates undifferentiated representations; lower similarity with higher variance suggests task-specific learning. MAML’s gradients exhibit high mutual similarity (mean 0.884, std 0.047), confirming that coupled optimization produces undifferentiated representations. STRIDE shows lower mean (0.749) with higher variance (0.116), demonstrating that temporal separation preserves domain-specific optimization directions (Figure 7, Table 8).



| Metric                       | MAML   | STRIDE        | Ratio       |
|------------------------------|--------|---------------|-------------|
| <i>Qwen2.5-0.5B-Instruct</i> |        |               |             |
| <b>Weight Sparsity</b>       | 1.55%  | <b>10.01%</b> | 6.5× higher |
| <b>L2 Norm</b>               | 0.575  | <b>0.400</b>  | 30% lower   |
| <b>L1 Norm</b>               | 2512.1 | <b>866.1</b>  | 65% lower   |
| <b>Max Weight</b>            | 0.0017 | <b>0.0031</b> | 1.8× higher |

Table 7: LoRA weight analysis across 6 OOD tasks. STRIDE exhibits **6.5× higher sparsity** with lower norms but higher peak magnitudes-focused adaptation rather than diffuse updates.

| Method                       | Mean         | Std          | Min   | Max   |
|------------------------------|--------------|--------------|-------|-------|
| <i>Qwen2.5-0.5B-Instruct</i> |              |              |       |       |
| MAML-en-LLM                  | 0.884        | 0.047        | 0.770 | 0.976 |
| <b>STRIDE (proposed)</b>     | <b>0.749</b> | <b>0.116</b> | 0.476 | 0.940 |

Table 8: Pairwise gradient cosine similarity across 6 OOD tasks. **Lower mean with higher variance** indicates better task-specific differentiation.

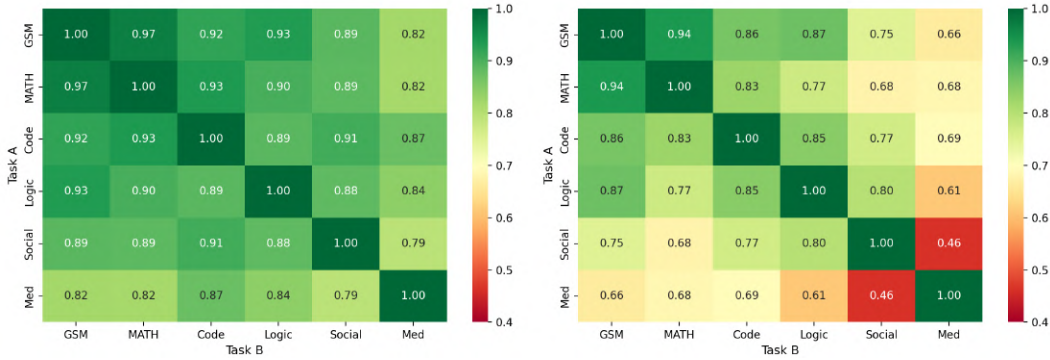


Figure 7: **Gradient Conflict Comparison.** MAML-en-LLM (left) shows uniformly high similarity; STRIDE (right) exhibits wider variance with mathematics tasks clustering tightly (GSM–MATH: 0.94) while distant pairs diverge (Social–Medical: 0.46).

### B.5.2 LORA WEIGHT SPARSITY AND STRUCTURE

We analyze LoRA weight properties across six OOD tasks to characterize optimization dynamics. STRIDE exhibits **6.5× higher sparsity**, with 30% lower L2 and 65% lower L1 norms. This indicates selective parameter modifications: fewer parameters carry the adaptation signal with higher peak magnitudes, consistent with temporal separation enabling task-focused updates (Table 7).

### B.5.3 VISUAL EVIDENCE: ADAPTATION SIGNATURES

We visualize LoRA parameter updates for Layer 12 across six OOD tasks by computing  $\Delta W = \frac{\alpha}{r} B A$ . MAML produces “barcode-like” patterns with identical weights across tasks (Figure 9: vertical stripes). STRIDE produces distinct “signatures” for each domain, visually confirming that temporal separation enables genuine task-specific specialization.

### B.5.4 TASK-SEMANTIC ALIGNMENT VERIFICATION

We verify that Stage 2 preserves task semantics by computing pairwise cosine similarity between aligned LoRA parameters and VAE conditioning latents. Mathematics tasks exhibit 0.949 row / 0.526 latent similarity, while JAMA (medical) shows near-zero similarity with math domains, confirming domain-appropriate separation.

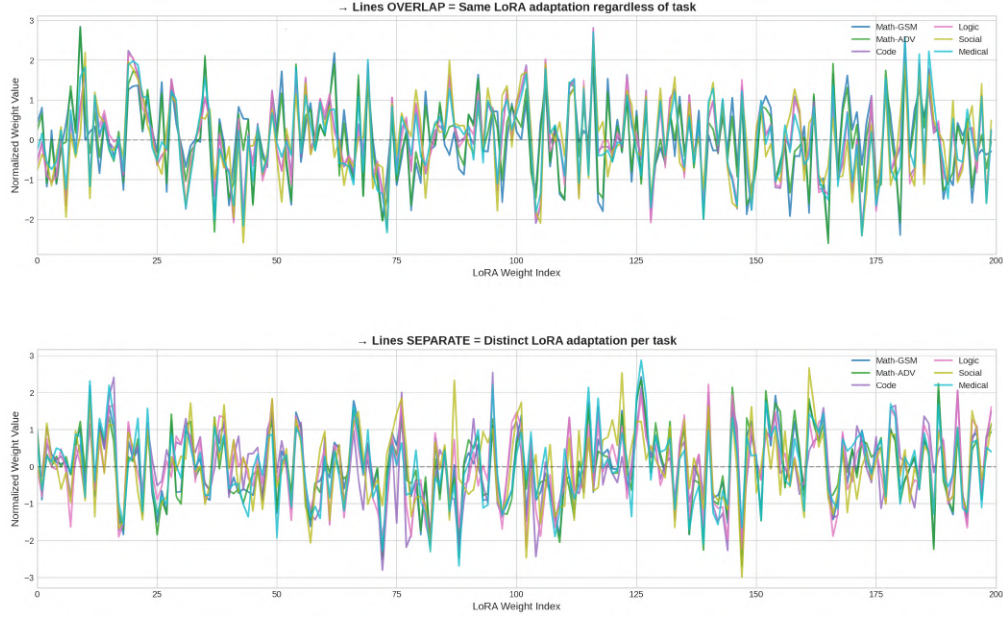


Figure 8: **LoRA Weight Pattern Visualization across 6 OOD Tasks.** MAML-en-LLM (top) shows nearly identical weight patterns—lines overlap—for all tasks, indicating a generic “barcode-like” adaptation that fails to specialize. STRIDE (bottom) displays clearly separated “signatures” for each domain (e.g., Code vs. Medical vs. Math), proving that temporal separation unlocks genuine task-specific parameter updates.

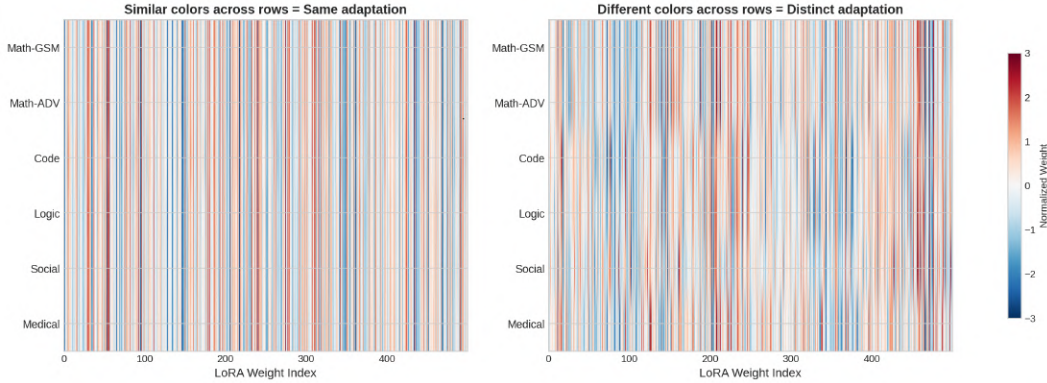


Figure 9: **LoRA Weight Heatmap.** MAML-en-LLM (left) shows uniform vertical stripes—identical parameters across tasks (“compromise”). STRIDE (right) shows varied patterns, demonstrating task-specific adaptations.

## C PHASE I: LEARNING

### C.1 STAGE 1: KNOWLEDGE GAP ASSESSMENT (KGA)

Stage I addresses a fundamental challenge in LLM adaptation: *how do we systematically identify knowledge gaps and generate optimally-sequenced training materials that respect the temporal structure of human learning?* Our solution, the **Knowledge Gap Assessment (KGA)** stage, addresses this through an automated multi-agent pipeline that generates adaptive teaching materials from unstructured domain text. The pipeline synthesizes techniques from recent advances in question

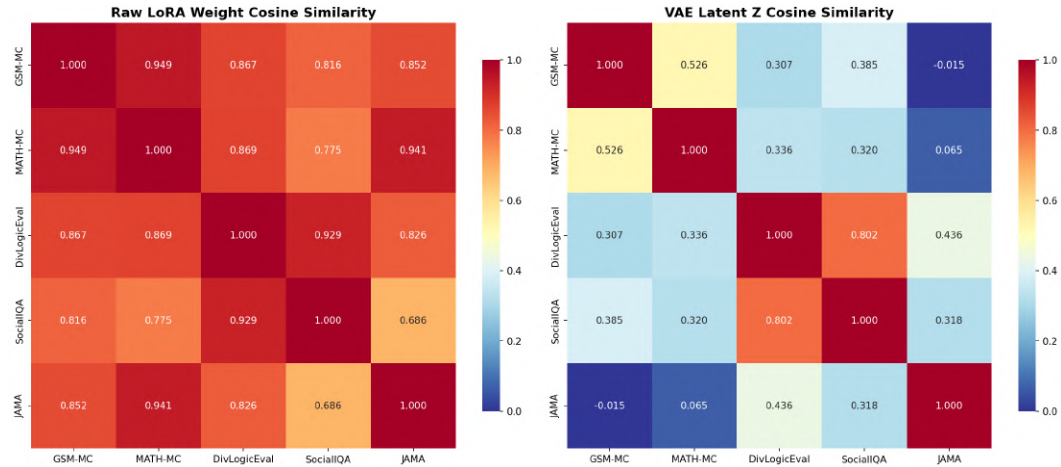


Figure 10: **Pairwise Similarity of Aligned LoRA Adapters.** Similar domains cluster (GSM–MATH: 0.949 raw, 0.526 latent) while distant domains separate (JAMA–Math: near-zero latent), confirming semantic preservation.

| SeRL adaptation on Qwen2.5-0.5B-Instruct |        |              |         |
|------------------------------------------|--------|--------------|---------|
| In-Domain                                | ID Acc | OOD          | OOD Acc |
| ARC-c                                    | 22.44  | GSM-8K       | 23.96   |
| ARC-e                                    | 23.99  | MATH         | 24.66   |
| HellaSwag                                | 25.04  | DivLogicEval | 27.44   |
| BoolQ                                    | 44.74  | SocialIQA    | 32.96   |
| PIQA                                     | 96.70  | CodeMMLU     | 26.26   |
| WinoGrande                               | 49.57  | JAMA         | 12.38   |

Table 9: Poor performance (most scores <30%), demonstrate the collapse of self-evolution when models lack sufficient foundational expertise to generate informative training curricula.

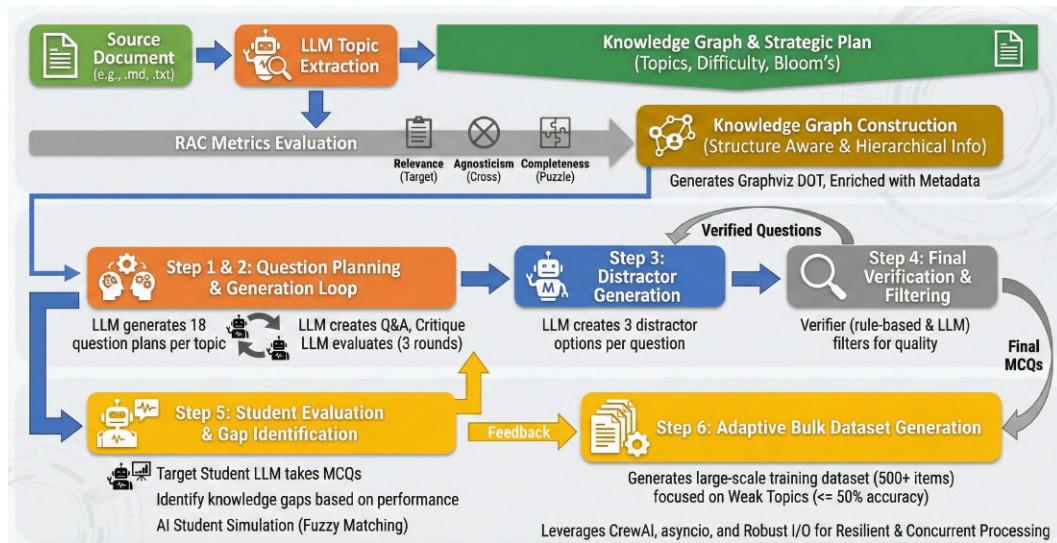


Figure 11: Stage 1 : Knowledge Gap Assessment

generation Wan et al. (2024b); Chen et al. (2025b), distractor synthesis Chen et al. (2024) and structure-aware training Zhang et al. (2025b).

### C.1.1 PIPELINE OVERVIEW

KGA implements a 6-step pipeline that transforms raw domain text into an adaptive learning curriculum:

1. **Strategic Planning:** Decompose unstructured text into structured pedagogical topics
2. **Knowledge Graph Construction:** Build semantic concept graphs with enriched edge attributes
3. **Initial MCQ Generation:** Generate Bloom’s-Taxonomy-aligned questions per topic
4. **Distractor Generation:** Synthesize plausible distractors with self-refinement
5. **Simulated Student Assessment:** Probe model competence via fuzzy-matched AI responses
6. **Adaptive Content Generation:** Allocate resources proportionally to identified weak topics

### C.1.2 STEP 1: STRATEGIC PLANNING

The first step transforms unstructured domain content into a structured topic hierarchy, following the structure-aware decomposition approach of Zhang et al. (2025b).

**Process.** Given a raw text document  $\mathcal{D}$ , a planning agent extracts a topic list  $\mathcal{T} = \{t_1, t_2, \dots, t_K\}$  ordered by pedagogical precedence:

$$\mathcal{T} = \text{PlanAgent}(\mathcal{D}) \quad \text{s.t.} \quad t_i \prec t_j \Rightarrow \text{prereq}(t_i, t_j) \quad (17)$$

where  $\prec$  denotes prerequisite ordering. Topics are normalized to dictionary format  $\{\text{“topic”}: t_i, \text{“difficulty”}: d_i, \text{“prerequisites”}: [t_j, \dots]\}$ .

**Robust Parsing.** LLM outputs frequently contain malformed JSON (missing quotes, trailing commas). We employ a multi-stage repair mechanism:

1. Attempt standard JSON parsing
2. Fall back to JSON5 parsing (permissive syntax)
3. Apply regex-based quote repair for unquoted keys
4. Normalize simple lists to dictionary format

**Cognitive Alignment.** The prerequisite ordering ensures that foundational concepts are addressed before dependent ones, implementing Bloom’s mastery learning principle at the curriculum level. This contrasts with random sampling approaches that may introduce concepts before learners are prepared.

### C.1.3 STEP 2: KNOWLEDGE GRAPH CONSTRUCTION

The second step constructs a directed knowledge graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  representing concept relationships, with enriched edge attributes for adaptive learning.

**Graph Generation.** A graph synthesis agent produces a DOT-format graph from the source text:

$$\mathcal{G}_{\text{raw}} = \text{GraphAgent}(\mathcal{D}, \mathcal{T}) \quad (18)$$

**Edge Enrichment Algorithm.** Raw graphs lack the metadata required for adaptive learning. We programmatically enrich edges using semantic inference:

- 1: **for** each edge  $(u, v) \in \mathcal{E}$  **do**
- 2:   **if** “subset”  $\in \text{label}(u)$  **and** “universal”  $\in \text{label}(v)$  **then**
- 3:      $\text{relation}(u, v) \leftarrow \text{“prereq”}$ ;  $\text{op}(u, v) \leftarrow \text{“generalize”}$
- 4:   **else if** “example”  $\in \text{label}(u)$  **then**
- 5:      $\text{relation}(u, v) \leftarrow \text{“instantiation”}$ ;  $\text{op}(u, v) \leftarrow \text{“specialize”}$
- 6:   **else**

```

7:      relation( $u, v$ )  $\leftarrow$  "related"; op( $u, v$ )  $\leftarrow$  "associate"
8:  end if
9:  Apply defaults: weight( $u, v$ )  $\leftarrow$  0.7, difficulty_delta( $u, v$ )  $\leftarrow$  0
10: Append visualization: label( $u, v$ )  $\leftarrow$  "w=weight"
11: end for

```

**Output.** The enriched graph  $\mathcal{G}$  captures:

- **Nodes:** Concepts with difficulty ratings and category labels
- **Edges:** Directed relationships with semantic type, weight and difficulty delta
- **Visualization:** Rendered PNG with edge weights for human inspection

#### C.1.4 STEP 3: INITIAL MCQ GENERATION

The third step generates a "seed" quiz spanning all strategic topics, with questions distributed across Bloom’s Taxonomy cognitive levels.

**Bloom’s Taxonomy Distribution.** Following Wan et al. (2024b)’s RACAR framework for quality assessment, we request questions at specified cognitive levels:

$$\text{distribution} = \{\text{Apply} : 1, \text{Analyze} : 1, \text{Create} : 1\} \quad (19)$$

This ensures questions test genuine understanding rather than surface-level recall.

**Concurrent Generation with Rate Limiting.** To maximize throughput while respecting API constraints, we employ an asynchronous generation strategy:

$$\text{AsyncRateLimiter}(\text{interval} = 1.5s, \text{max\_concurrency} = 8) \quad (20)$$

with exponential backoff and jitter for rate limit and timeout recovery.

**Anticipatory Planning.** Following the LEPA approach Chen et al. (2025b), questions are generated with explicit reasoning chains that anticipate learner misconceptions:

1. Identify the core concept being tested
2. Enumerate common misconceptions or partial understandings
3. Construct the question to distinguish mastery from near-miss understanding
4. Generate rationale linking the question to prerequisite concepts

#### C.1.5 STEP 4: DISTRACTOR GENERATION

The fourth step transforms raw questions into validated multiple-choice items by generating plausible incorrect options.

**Self-Refinement Approach.** Following Chen et al. (2024)’s MCQG-SRefine methodology, distractor generation incorporates iterative refinement:

$$\text{distractors}^{(k+1)} = \text{Refine}(\text{distractors}^{(k)}, \text{feedback}^{(k)}) \quad (21)$$

where feedback identifies distractors that are too obviously wrong, too similar to the correct answer, or grammatically inconsistent.

**Quality Validation via RACAR Metrics.** Each generated item is evaluated against the RACAR criteria Wan et al. (2024b):

- **Relevance:** Question addresses specified topic
- **Agnosticism:** Question is self-contained (no external context required)
- **Completeness:** All necessary information is provided
- **Accuracy:** Correct answer is unambiguously correct
- **Reasonableness:** Distractors are plausible but clearly incorrect

**Option Normalization.** Post-generation processing ensures consistent formatting:

1. Verify correct answer presence in options
2. Pad to 4 options if fewer exist
3. Randomly shuffle option order (preventing position bias)
4. Sanitize text (remove artifacts, normalize whitespace)

#### C.1.6 STEP 5: SIMULATED STUDENT ASSESSMENT

The fifth step identifies the model’s Zone of Proximal Development by simulating student responses to the generated quiz.

**Student Model.** A separate LLM (e.g., Llama 3.1 8B Instruct) acts as a simulated student, attempting each question without access to the answer key:

$$\text{response}_i = \text{StudentLLM}(q_i, \text{options}_i) \quad (22)$$

**Fuzzy Matching Evaluation.** Student responses are often conversational rather than exact option matches. We employ fuzzy string matching to determine the selected answer:

$$\text{selected}_i = \arg \max_{o \in \text{options}_i} \text{fuzzy.partial\_ratio}(\text{response}_i, o) \quad (23)$$

**Performance Analysis by Topic.** For each topic  $t \in \mathcal{T}$ , we compute:

$$\text{accuracy}(t) = \frac{\sum_{q \in Q_t} \mathbb{1}[\text{selected}_q = \text{correct}_q]}{|Q_t|} \quad (24)$$

where  $Q_t$  denotes questions associated with topic  $t$ .

**ZPD Identification.** Topics are partitioned into zones based on performance:

$$\text{Weak} = \{t : \text{accuracy}(t) \leq 0.5\}, \quad \text{Strong} = \{t : \text{accuracy}(t) > 0.5\} \quad (25)$$

Weak topics represent the upper boundary of the current Zone of Proximal Development-areas where the model fails independently but could succeed with appropriate training.

#### C.1.7 STEP 6: ADAPTIVE CONTENT GENERATION

The final step generates a large-scale training dataset with resources allocated proportionally to identified weaknesses, implementing Vygotsky’s principle of scaffolding within the Zone of Proximal Development.

**Resource Allocation.** Given a target dataset size  $N$  and weak topic ratio  $\rho$  (default 0.70), we allocate:

$$N_{\text{weak}} = \lfloor \rho \cdot N \rfloor, \quad N_{\text{strong}} = N - N_{\text{weak}} \quad (26)$$

If 500 questions are needed with  $\rho = 0.70$ , approximately 350 target weak topics and 150 target strong topics.

**Micro-Batching.** To prevent context window overflow and enable granular error recovery, generation proceeds in batches:

$$\text{batches} = \left\lceil \frac{N_{\text{topic}}}{\text{batch\_size}} \right\rceil, \quad \text{batch\_size} = 10 \quad (27)$$

Each batch spawns an independent generation crew, enabling parallel processing with isolated failure domains.

**Dynamic Timeout Calculation.** To accommodate variable generation complexity:

$$\text{timeout} = \text{base} + (\text{per\_question\_time} \times \text{batch\_size}) \quad (28)$$



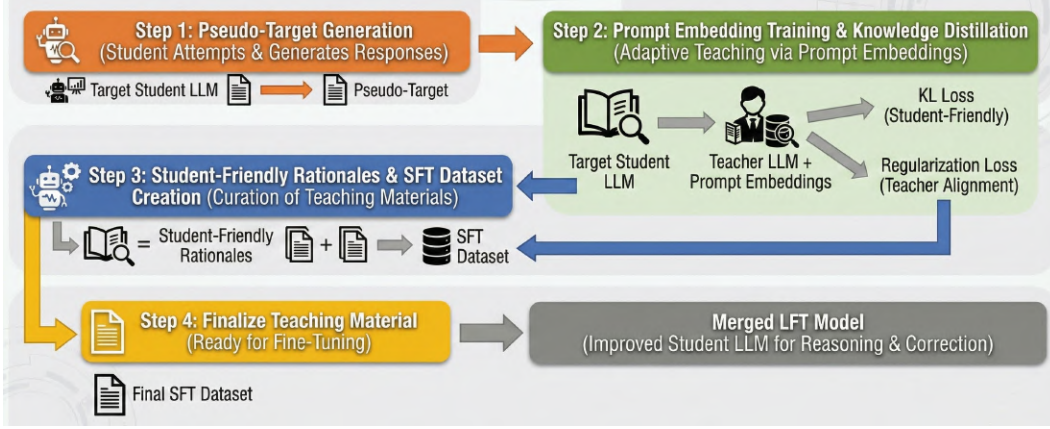


Figure 12: Stage 2 &amp; 3: Teaching Material Curation and Learning from Teacher

**Top-Up Mechanism.** If generated count falls short of target (due to filtering or agent errors), a second pass generates specifically the missing quantity, ensuring the curriculum is complete.

**Output.** The final adaptive dataset  $\mathcal{D}_{\text{KGA}}$  is structured as, for example like:

```
{
  "topic": "Integration by Parts",
  "difficulty": "medium",
  "bloom_level": "Apply",
  "question": "...",
  "options": ["A", "B", "C", "D"],
  "correct_answer": "B",
  "rationale": "...",
  "prerequisites": ["Product Rule", "Antiderivatives"]
}
```

## C.2 STAGE 2 & 3: TEACHING MATERIAL CURATION AND LEARNING FROM TEACHER

Following the Knowledge Graph-Augmented Learning stage (Stage 1), the next stages address a fundamental challenge in AI-assisted learning: *how do we curate teaching materials that match the student model’s cognitive capacity?* Human educators intuitively adjust their explanations to match student comprehension levels—a process that cognitive science has formalized through research on *adaptive teaching* Sweller (1988) and *faded scaffolding* Collins et al. (1989). Stages 2 and 3 operationalize these principles for LLM knowledge transfer. Related works include Kim et al. (2024); He et al. (2025); Kujanpää et al. (2025)

### C.2.1 PIPELINE OVERVIEW

The Teaching Material Curation and Learning from Teacher pipeline implements a four-step process (Figure 12), transforming raw teacher knowledge into student-calibrated training materials:

1. **Step 1: Pseudo-Target Generation** - The student model attempts problems from the curated dataset (Stage 1) and generates candidate responses, establishing baselines for current capability.
2. **Step 2: Prompt Embedding Training & Knowledge Distillation** - Learnable prompt embeddings are trained to condition the teacher model toward student-friendly output distributions through KL-divergence minimization.
3. **Step 3: Student-Friendly Rationale Creation & SFT Dataset Curation** - The prompted teacher generates explanations and rationales calibrated to student comprehension, which are curated into structured training examples.

4. **Step 4: Finalize Teaching Material** - Quality filtering and format standardization produce the final SFT dataset for student model training.

#### C.2.2 STEP 1: PSEUDO-TARGET GENERATION

The first step establishes a baseline characterization of the student model’s current capabilities through self-generated responses.

**Motivation from Exposure Bias.** Standard knowledge distillation trains students on teacher-generated text, creating a distribution mismatch: during inference, students generate from their own distribution, which differs from the teacher’s training distribution. This *exposure bias* Zhang et al. (2019) causes error accumulation in autoregressive generation. Using student-generated pseudo-targets during training aligns the training distribution with inference conditions.

**Generation Process.** For each problem  $x_i$  from the KGA-curated dataset  $\mathcal{D}_{\text{KGA}}$ , the student model  $S_\theta$  generates a candidate response:

$$\hat{y}_i = S_\theta(x_i) \sim q_\theta(\cdot|x_i) \quad (29)$$

where  $q_\theta$  denotes the student’s output distribution. Generation uses nucleus sampling Holtzman et al. (2020) appropriately to balance diversity with coherence. Importantly, this sampling does not propagate gradients-the pseudo-targets serve as conditioning for subsequent steps, not as direct training signal.

**Capability Assessment.** Pseudo-target quality provides a proxy for current student capability. For each response, we compute surface metrics (length, fluency) and optionally semantic similarity to reference solutions, creating a per-problem difficulty profile that informs later curriculum sequencing.

#### C.2.3 STEP 2: PROMPT EMBEDDING TRAINING FOR ADAPTIVE TEACHING

The core innovation of Stage 2-3 is training learnable prompt embeddings that condition the teacher model to generate student-friendly outputs-implementing adaptive teaching through learned input modulation.

**Prompt Tuning Architecture.** Following efficient fine-tuning principles Lester et al. (2021), we prepend  $m$  learnable prompt tokens  $P = \{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_m\}$  to the teacher’s input, where each  $\mathbf{p}_i \in \mathbb{R}^d$  matches the token embedding dimension. The teacher remains frozen; only the prompt embeddings are trained:

$$T(x|P) = T_\phi(\text{concat}(P, \text{embed}(x))) \quad (30)$$

where  $T_\phi$  denotes the frozen teacher with parameters  $\phi$  and  $\text{embed}(\cdot)$  is the input embedding layer.

**Student-Guided Prompt Training.** The prompt embeddings are trained to minimize distributional distance between teacher output (conditioned on prompts) and student output. Using the pseudo-targets  $\hat{y}$  as conditioning, we define:

$$\mathcal{L}_{\text{adapt}} = D_{\text{KL}}\left(p(\hat{y}|P, x; \phi) \parallel q_\theta(\hat{y}|x)\right) \quad (31)$$

where  $p(\cdot|P, x; \phi)$  is the prompted teacher distribution and  $q_\theta(\cdot|x)$  is the student distribution. Minimizing this reverse KL divergence encourages the teacher to place probability mass where the student already has reasonable coverage-effectively lowering its “explanatory register” to match student comprehension.

**Regularization for Stable Teaching.** Early in training, abrupt prompt-induced distribution shifts can destabilize the teacher’s generation quality. We regularize by encouraging the prompted teacher to remain close to the unprompted teacher:

$$\mathcal{L}_{\text{reg}} = D_{\text{KL}}\left(p(\hat{y}|P, x; \phi) \parallel p(\hat{y}|x; \phi)\right) \quad (32)$$

This regularization is annealed during training, starting at weight  $\lambda_{\text{reg}} = 1.0$  and linearly decaying to 0 over the training period. The combined prompt training objective is:

$$\mathcal{L}_{\text{prompt}} = \mathcal{L}_{\text{adapt}} + \lambda_{\text{reg}}(t) \cdot \mathcal{L}_{\text{reg}}, \quad \text{where} \quad \lambda_{\text{reg}}(t) = 1 - \frac{t}{T} \quad (33)$$



**Cognitive Interpretation.** The trained prompts implement adaptive teaching without modifying the teacher’s knowledge. They can be understood as learned “teaching style” vectors that translate the teacher’s expert-level knowledge into student-accessible forms-analogous to how human teachers adjust vocabulary and abstraction level based on student feedback.

#### C.2.4 STEP 3: STUDENT-FRIENDLY RATIONALE CURATION

With trained prompts that induce student-friendly teacher behavior, Step 3 generates and curates the training materials.

**Rationale Generation.** For each problem  $x_i$ , the prompted teacher generates explanatory rationales:

$$r_i = T_\phi(x_i|P^*), \quad \text{where } P^* = \arg \min_P \mathcal{L}_{\text{prompt}} \quad (34)$$

These rationales exhibit key properties validated by the prompt training:

- *Comprehensibility*: Language and conceptual complexity matched to student capacity
- *Correctness*: Content accuracy preserved from original teacher knowledge
- *Scaffolding*: Step-by-step reasoning chains that externalize expert thought processes

**SFT Dataset Construction.** Rationales are formatted into instruction-following examples:

```
{
  "instruction": "Solve the following problem step-by-step.",
  "input": "<problem statement>",
  "output": "<student-friendly rationale>",
  "metadata": {
    "topic": "<topic_name>",
    "difficulty": "<bloom_level>",
    "source": "Stage23_Curation"
  }
}
```

**Quality Filtering via RACAR Criteria.** Following the quality assurance framework from Stage 1, generated rationales are evaluated against:

- **Relevance**: Rationale addresses the problem directly
- **Accuracy**: Mathematical and logical correctness verified
- **Completeness**: All steps necessary for solution are present
- **Accessibility**: Language complexity appropriate for student level
- **Reasoning**: Clear causal chain from premises to conclusion

#### C.2.5 STEP 4: FINALIZE TEACHING MATERIAL AND FINE-TUNING

The final step prepares validated materials for student model training.

**Dataset Composition.** The finalized SFT dataset combines:

- **KGA Materials** (Stage 1): Bloom’s-Taxonomy-aligned MCQ training examples
- **Student-Friendly Rationales** (Stage 2-3): Prompted teacher explanations
- **Difficulty Metadata**: Topic, complexity and prerequisite structure

This composition ensures training covers both factual knowledge (MCQs) and procedural reasoning (rationales).

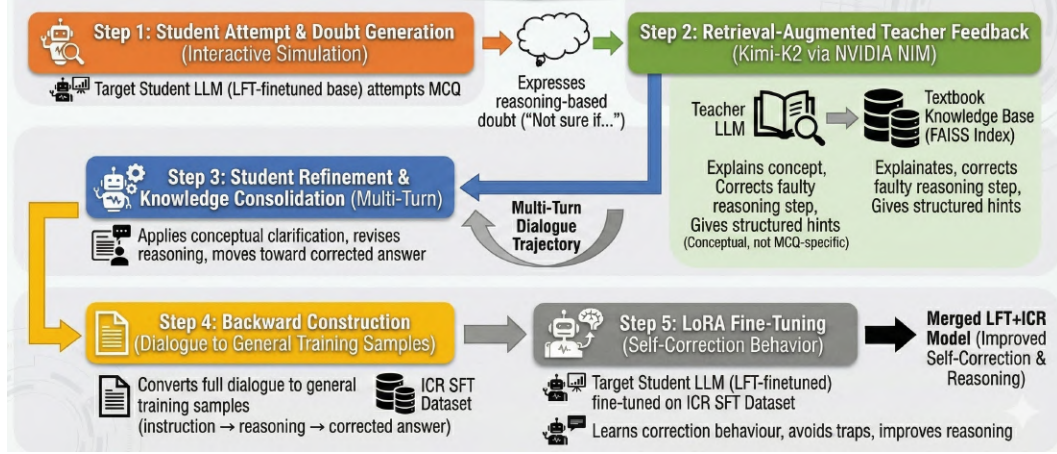


Figure 13: Stage 4: Interactive Concept Refinement and Doubt Clearance

**Student Fine-Tuning.** The student model is fine-tuned to minimize cross-entropy loss between its outputs and the curated rationales:

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x,r) \sim \mathcal{D}_{\text{SFT}}} \left[ \sum_{t=1}^{|r|} \log q_{\theta}(r_t | r_{<t}, x) \right] \quad (35)$$

This supervised fine-tuning phase trains the student to internalize the reasoning patterns demonstrated in teacher-generated rationales.

**Integration with Knowledge Distillation.** Optionally, fine-tuning can be augmented with online distillation, where the student also matches the prompted teacher’s output distribution during training:

$$\mathcal{L}_{\text{combined}} = \mathcal{L}_{\text{SFT}} + \alpha \cdot D_{\text{KL}}(q_{\theta}(\cdot | x) \parallel p(\cdot | P^*, x; \phi)) \quad (36)$$

The reverse KL term (student as proposal distribution) encourages mode-seeking behavior appropriate for generation tasks Gu et al. (2023).

### C.3 STAGE 4: INTERACTIVE CONCEPT REFINEMENT AND DOUBT CLEARANCE

Stages 1-3 establish a foundation of knowledge and reasoning patterns through passive learning-the student absorbs curated materials without active inquiry. Stage 4 completes Phase 1 by introducing *interactive learning*, where the student actively generates questions, receives targeted feedback and refines understanding through multi-turn dialogue. This parallels the human educational practice of tutorial sessions and doubt clarification, operationalizing key principles from Socratic pedagogy Reich (2003) and interactive concept learning research Chi & Wylie (2014).

#### C.3.1 PIPELINE OVERVIEW

The Interactive Concept Refinement pipeline implements a five-step iterative process:

1. **Step 1: Student Attempt and Doubt Generation** - The student attempts problems and explicitly articulates areas of confusion or uncertainty.
2. **Step 2: Retrieval-Augmented Teacher Feedback** - A knowledge base (textbook corpus) is queried to provide contextually relevant information, which grounds the teacher’s response to student doubts.
3. **Step 3: Student Refinement and Knowledge Consolidation** - The student processes teacher feedback and refines its response, potentially generating follow-up doubts for multi-turn interaction.

4. **Step 4: Backward Construction** - Successful interaction trajectories are transformed into training examples using backward construction, maximizing data utility.
5. **Step 5: ICR-Enhanced Fine-Tuning** - The curated ICR dataset is used to fine-tune the student via LoRA, instilling interactive reasoning patterns.

### C.3.2 STEP 1: STUDENT ATTEMPT AND DOUBT GENERATION

The student model first attempts a problem independently, then generates explicit doubt statements identifying areas of uncertainty.

**Initial Attempt.** Given a problem  $x$  and the student model  $S_\theta$  (post-Stage 2-3 fine-tuning), the student generates a draft response:

$$y^{(0)} = S_\theta(x) \sim q_\theta(\cdot|x) \quad (37)$$

This draft exhibits the student’s current capabilities and reveals knowledge gaps through errors or uncertainties.

**Structured Doubt Expression.** The student then generates a doubt statement conditioned on its own attempt:

$$d = S_\theta(\langle \text{DOUBT\_PROMPT} \rangle, x, y^{(0)}) \quad (38)$$

where  $\langle \text{DOUBT\_PROMPT} \rangle$  instructs the student to articulate specific points of confusion. The doubt template follows a structured format:

I am uncertain about [specific concept/step].  
 My current understanding is [student's interpretation].  
 I am confused because [specific point of confusion].

This structured format ensures doubts are actionable and provide the teacher with sufficient context for targeted feedback.

**Doubt Classification.** Generated doubts are classified by type to inform retrieval strategy:

- **Conceptual:** Fundamental misunderstanding of a concept
- **Procedural:** Uncertainty about steps or methods
- **Application:** Confusion about when/how to apply known concepts
- **Connection:** Difficulty relating new information to prior knowledge

### C.3.3 STEP 2: RETRIEVAL-AUGMENTED TEACHER FEEDBACK

The teacher’s response is grounded in retrieved knowledge to ensure accuracy and relevance.

**Knowledge Base Construction.** A textbook corpus  $\mathcal{K}$  is indexed using dense retrieval. Each document  $k_i \in \mathcal{K}$  is embedded into a vector representation:

$$\mathbf{e}_i = \text{Encoder}(k_i) \in \mathbb{R}^d \quad (39)$$

We use a contrastive-trained encoder (e.g., Contriever Izacard et al. (2022)) and index embeddings using FAISS Johnson et al. (2019) for efficient approximate nearest neighbor search.

**Doubt-Conditioned Retrieval.** Given the student’s doubt  $d$ , we retrieve the  $k$  most relevant knowledge segments:

$$\mathcal{R}(d) = \text{TopK}\left(\{\langle \text{Encoder}(d), \mathbf{e}_i \rangle : k_i \in \mathcal{K}\}, k\right) \quad (40)$$

Retrieved segments provide authoritative context for the teacher’s response.

**Grounded Teacher Response.** The teacher generates feedback conditioned on the retrieved context, student doubt and original problem:

$$f = T_\phi(\mathcal{R}(d), d, x, y^{(0)}) \quad (41)$$

This retrieval augmentation ensures the teacher’s feedback is factually grounded and directly addresses the student’s specific confusion points-implementing what educational researchers call “just-in-time” feedback.

**Response Structure.** Teacher feedback follows a pedagogical structure:

1. **Acknowledge:** Validate the student’s confusion and restate the problem
2. **Clarify:** Directly address the misconception with correct information
3. **Connect:** Link the clarification to concepts the student already understands
4. **Guide:** Provide hints toward the solution without giving it away

### C.3.4 STEP 3: STUDENT REFINEMENT AND MULTI-TURN DIALOGUE

The student processes teacher feedback and generates a refined response, with potential for iterative clarification.

**Response Refinement.** After receiving feedback  $f$ , the student generates an improved response:

$$y^{(t+1)} = S_\theta(x, y^{(t)}, d^{(t)}, f^{(t)}) \quad (42)$$

where  $t$  indexes the dialogue turn. The student integrates teacher guidance while maintaining coherence with its previous attempts.

**Iterative Doubt Clarification.** If the student remains uncertain after initial feedback, it may generate follow-up doubts:

$$d^{(t+1)} = S_\theta(\langle \text{FOLLOWUP\_DOUBT} \rangle, x, y^{(t+1)}, f^{(t)}) \quad (43)$$

Dialogue continues until the student indicates understanding (via a termination signal) or a maximum turn limit  $T_{\max}$  is reached. Empirically, we find  $T_{\max} = 3$  balances thoroughness with efficiency.

**Dialogue Trajectory.** A complete ICR session produces a trajectory:

$$\tau = (x, y^{(0)}, d^{(1)}, f^{(1)}, y^{(1)}, \dots, d^{(T)}, f^{(T)}, y^{(T)}) \quad (44)$$

This trajectory captures the complete learning interaction and becomes the raw material for training data construction.

### C.3.5 STEP 4: BACKWARD CONSTRUCTION FOR DATA SYNTHESIS

Following the Learn-by-Interact paradigm Gui et al. (2024), we extract multiple training examples from each dialogue trajectory through backward construction.

**Sub-Trajectory Extraction.** From a complete trajectory  $\tau$  with  $T$  turns, we extract sub-trajectories of varying lengths:

$$\tau_{i:j} = (y^{(i)}, d^{(i+1)}, f^{(i+1)}, \dots, y^{(j)}), \quad 0 \leq i < j \leq T \quad (45)$$

This yields  $\binom{T+1}{2}$  potential sub-trajectories-a quadratic increase in training data from each interaction session.

**Instruction Synthesis.** For each sub-trajectory, we generate corresponding task instructions through two mechanisms:

*Summarization:* An LLM summarizes the sub-trajectory into a concise instruction describing what the sequence accomplishes:

$$I_{\text{sum}} = \text{LLM}(\langle \text{SUMMARIZE} \rangle, \tau_{i:j}) \quad (46)$$

*Abstraction:* For complete trajectories, we abstract the overall learning goal, correcting for any misalignment between original task and actual trajectory behavior:

$$I_{\text{abs}} = \text{LLM}(\langle \text{ABSTRACT\_GOAL} \rangle, x, \tau) \quad (47)$$

**Quality Filtering via Committee Verification.** Synthesized instruction-trajectory pairs undergo quality filtering through a committee of LLMs. A pair  $(I', \tau')$  is retained only if all committee members judge it to be:

- **Coherent:** Actions follow logically from prior states
- **Natural:** Dialogue flow resembles authentic tutoring interactions
- **Reasonable:** Steps taken are sensible approaches to the problem
- **Aligned:** Trajectory accomplishes the stated instruction

### C.3.6 STEP 5: ICR-ENHANCED FINE-TUNING

The curated ICR dataset instills interactive reasoning patterns in the student through LoRA-based fine-tuning.

**Dataset Composition.** The final ICR training dataset combines:

- **Full trajectories:** Complete multi-turn interactions with final solutions
- **Doubt-resolution pairs:**  $(d, f, y_{\text{improved}})$  triplets for targeted doubt clarification
- **Sub-trajectory skills:** Extracted skills from backward construction

**LoRA Fine-Tuning.** We apply Low-Rank Adaptation Hu et al. (2022a) to the student model to minimize:

$$\mathcal{L}_{\text{ICR}} = -\mathbb{E}_{(I', \tau') \sim \mathcal{D}_{\text{ICR}}} \left[ \sum_{t=1}^{|\tau'|} \log q_{\theta}(\tau'_t | \tau'_{<t}, I') \right] \quad (48)$$

LoRA is preferred over full fine-tuning to preserve knowledge acquired in Stages 1-3 while adding interactive reasoning capabilities.

## D PHASE II : MASTERY

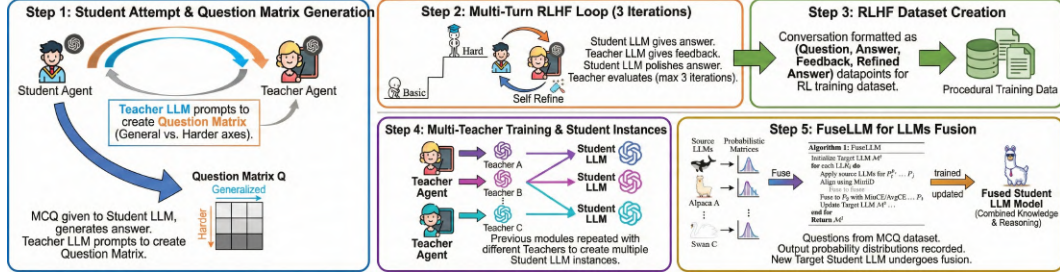


Figure 14: Stages 5 & 6: Feedback-Driven Practice and Multi-Teacher Knowledge Distillation

Having completed the Learning phase (Stages 1-4), where the student acquires foundational knowledge through scaffolded instruction, the Mastery phase (Stages 5-7) addresses a fundamental question: *how do we transition from knowledge acquisition to procedural fluency through deliberate practice?* Human learners consolidate understanding through active problem-solving with corrective feedback and often seek diverse perspectives from multiple instructors to avoid overfitting to a single teaching style. Stages 5 and 6 operationalize these principles for LLM adaptation.

### D.1 STAGE 5: FEEDBACK-DRIVEN PRACTICE

Stage 5 implements deliberate practice through a structured teacher-student interaction loop, drawing inspiration from the related frameworks such as Zhang et al. (2024a); Li et al. (2024) for teacher-student progressive learning with iterative refinement.

#### D.1.1 PIPELINE OVERVIEW

The Feedback-Driven Practice pipeline implements a four-step iterative process:

- Step 1: Student Attempt and Question Matrix Generation** - The student attempts problems from the curated dataset and the teacher generates a structured question matrix spanning generalization and difficulty axes.
- Step 2: Multi-Turn Feedback Loop** - For each question, the student generates an answer, receives teacher feedback, refines its response and iterates until correctness or a maximum iteration limit.
- Step 3: Procedural Training Data Creation** - Successful interaction trajectories are formatted into structured training examples capturing the full feedback-refinement process.
- Step 4: Reinforcement Learning Fine-Tuning** - The curated dataset is used to train the student with RL objectives that reward both correctness and the ability to learn from feedback.

#### D.1.2 STEP 1: STUDENT ATTEMPT AND QUESTION MATRIX GENERATION

The first step establishes the practice curriculum through systematic question generation.

**Initial Student Attempt.** Given a problem  $q^{(0,0)}$  from the MCQ dataset  $\mathcal{D}_{\text{KGA}}$ , the student model  $S_\theta$  generates an initial response:

$$a^{(0,0)} = S_\theta(q^{(0,0)}) \sim q_\theta(\cdot | q^{(0,0)}) \quad (49)$$

This response reveals the student’s current capability level on the specific problem type.

**Teacher-Generated Question Matrix.** Based on the student’s response quality, the teacher LLM  $T_\phi$  generates a structured question matrix  $\mathbf{Q} \in \mathbb{R}^{(n+1) \times (m+1)}$  with two axes:

- **Generalization axis** (horizontal): Questions  $\{q^{(i,0)}, q^{(i,1)}, \dots, q^{(i,m)}\}$  of similar difficulty but varying scope and context
- **Difficulty axis** (vertical): Questions  $\{q^{(0,j)}, q^{(1,j)}, \dots, q^{(n,j)}\}$  of progressively increasing complexity

The question matrix is formally defined as:

$$\mathbf{Q} = \begin{bmatrix} q^{(0,0)} & q^{(0,1)} & \dots & q^{(0,m)} \\ q^{(1,0)} & q^{(1,1)} & \dots & q^{(1,m)} \\ \vdots & \vdots & \ddots & \vdots \\ q^{(n,0)} & q^{(n,1)} & \dots & q^{(n,m)} \end{bmatrix} \quad (50)$$

where  $q^{(0,0)}$  is the original problem and each subsequent entry is generated by the teacher through:

$$q^{(i,j+1)} = T_\phi(\langle \text{GENERALIZE} \rangle, q^{(i,j)}) \quad (\text{generalization}) \quad (51)$$

$$q^{(i+1,j)} = T_\phi(\langle \text{COMPLICATE} \rangle, q^{(i,j)}) \quad (\text{difficulty increase}) \quad (52)$$

**ZPD-Calibrated Difficulty.** The teacher adjusts difficulty progression based on student performance on the initial problem. If the student’s response  $a^{(0,0)}$  is correct, the matrix emphasizes harder problems ( $n > m$ ); if incorrect, it emphasizes generalization to consolidate understanding ( $m > n$ ).

#### D.1.3 STEP 2: MULTI-TURN FEEDBACK LOOP

For each question in the matrix, a structured feedback-refinement cycle operates until convergence.

**Iterative Refinement Protocol.** For each question  $q^{(i,j)} \in \mathbf{Q}$ , the following loop executes:

```

1: Initialize:  $t \leftarrow 0$ , attempt  $a_0 = S_\theta(q^{(i,j)})$ 
2: repeat
3:   Teacher evaluates:  $f_t = T_\phi(\langle \text{EVALUATE} \rangle, q^{(i,j)}, a_t)$ 
4:   if  $f_t$  indicates correctness then
5:     Break with success
6:   else
7:     Student refines:  $a_{t+1} = S_\theta(\langle \text{REFINE} \rangle, q^{(i,j)}, a_t, f_t)$ 
8:      $t \leftarrow t + 1$ 
9:   end if
10: until  $t = T_{\max}$  (typically  $T_{\max} = 3$ )

```

**Feedback Structure.** Teacher feedback  $f_t$  follows a pedagogical format:

```

{
  "is_correct": false,
  "error_type": "procedural|conceptual|computational",
  "specific_error": "<identification of the mistake>",
  "hint": "<guidance without revealing full answer>",
  "relevant_concept": "<concept to review>"
}

```

This structured feedback enables the student to identify the nature of its error and apply targeted improvements.

**Refinement Mechanism.** The student’s refinement process incorporates the feedback signal:

$$a_{t+1} = S_\theta(q^{(i,j)}, a_t, f_t) \quad (53)$$

The conditioning on both the previous attempt  $a_t$  and feedback  $f_t$  encourages the student to correct specific errors rather than regenerating from scratch.

**Progressive Matrix Traversal.** The question matrix is traversed in a structured order that respects prerequisite relationships:

1. Complete all generalization variants at difficulty level 0:  $q^{(0,0)} \rightarrow q^{(0,1)} \rightarrow \dots \rightarrow q^{(0,m)}$
2. Advance to the next difficulty level and repeat:  $q^{(1,0)} \rightarrow q^{(1,1)} \rightarrow \dots$
3. Continue until all matrix entries are processed

This traversal ensures foundational understanding at each difficulty level before advancing to harder problems—a direct implementation of Bloom’s mastery learning principle.

#### D.1.4 STEP 3: PROCEDURAL TRAINING DATA CREATION

Interaction trajectories are formatted into structured training examples that capture the complete feedback-refinement process.

**Trajectory Recording.** For each question  $q^{(i,j)}$ , the complete interaction trajectory is recorded:

$$\tau^{(i,j)} = (q^{(i,j)}, a_0, f_0, a_1, f_1, \dots, a_{T^*}, f_{T^*}) \quad (54)$$

where  $T^*$  is the iteration at which correctness is achieved (or  $T_{\max}$  if not achieved).

**Training Example Formats.** We create multiple training example formats to capture different aspects of the learning process:

*Question-Answer Pairs (QA Dataset):* Direct mappings from questions to final correct answers:

$$\mathcal{D}_{QA} = \left\{ (q^{(i,j)}, a_{T^*}^{(i,j)}) : q^{(i,j)} \in \mathbf{Q}, \text{correct}(a_{T^*}^{(i,j)}) \right\} \quad (55)$$

*Feedback-Refinement Tuples (QAFR Dataset):* Complete trajectories including the refinement process:

$$\mathcal{D}_{QAFR} = \left\{ (q, a_t, f_t, a_{t+1}) : t \in \{0, \dots, T^* - 1\} \right\} \quad (56)$$

**Data Quality Criteria.** Training examples are filtered based on:

- **Convergence:** Only trajectories that eventually reach correct answers
- **Non-trivial refinement:** At least one feedback-refinement cycle occurred
- **Coherent progression:** Each refinement shows improvement toward the correct answer

#### D.1.5 STEP 4: REINFORCEMENT LEARNING FINE-TUNING

The procedural training data is used to train the student using a combined supervised and RL objective.

**Supervised Component.** Following the training objective from YODA Zhang et al. (2024a):

$$\begin{aligned} \mathcal{L}(\phi) = & - \left( \mathbb{E}_{(q,a^*) \sim \mathcal{D}_{QA}} [\log \tau_\phi(a^*|q)] + \right. \\ & \left. \mathbb{E}_{(q,a,f,r) \sim \mathcal{D}_{QAFR}} [\log \tau_\phi(r|f,a,q) + \log \tau_\phi(f|a,q)] \right) \end{aligned} \quad (57)$$

This objective trains the student to: (1) produce correct answers directly, (2) generate appropriate feedback given an answer and (3) refine answers in response to feedback.

**Reinforcement Learning Enhancement.** To further improve the student’s ability to learn from feedback, we apply reinforcement learning with a reward based on answer improvement:

$$R(a_{t+1}, a_t, q) = \mathbb{1}[\text{quality}(a_{t+1}) > \text{quality}(a_t)] + \lambda \cdot \mathbb{1}[\text{correct}(a_{t+1})] \quad (58)$$

where  $\text{quality}(\cdot)$  is assessed by the teacher LLM and  $\lambda > 1$  provides additional reward for reaching the correct answer.



**ReST<sup>EM</sup> Training.** Following Rest<sup>EM</sup> Singh et al. (2024), we alternate between:

1. **E-step:** Generate multiple trajectories from the current policy, select high-reward trajectories
2. **M-step:** Fine-tune the student on selected high-reward trajectories

## D.2 STAGE 6: MULTI-TEACHER KNOWLEDGE DISTILLATION

Stage 6 addresses the limitation of single-teacher learning by integrating knowledge from multiple teacher LLMs, producing a student that generalizes better by avoiding overfitting to any single teaching style. This stage draws inspiration from FuseLLM Wan et al. (2024a) for multi-source knowledge integration.

### D.2.1 MOTIVATION: BEYOND SINGLE-TEACHER DISTILLATION

Traditional knowledge distillation trains a student to match a single teacher’s output distribution. While effective, this approach has limitations:

- **Teacher bias:** The student inherits biases and limitations specific to the teacher model
- **Narrow solution space:** The student learns only the teacher’s preferred problem-solving strategies
- **Overfitting risk:** Strong alignment with one teacher may reduce generalization to novel problems

Stage 6 addresses these limitations by treating Stages 1-5 as a *template* that can be instantiated with different teacher LLMs, then fusing the resulting student instances into a unified model.

### D.2.2 PIPELINE OVERVIEW

The Multi-Teacher Knowledge Distillation pipeline implements a three-step process:

1. **Step 1: Multi-Instance Student Training** - The complete Stage 1-5 pipeline is repeated with  $K$  different teacher LLMs, producing  $K$  student model instances.
2. **Step 2: Probabilistic Distribution Extraction** - Each student instance’s output probability distributions are recorded on a shared evaluation corpus.
3. **Step 3: Distribution Fusion and Target Student Training** - A new target student is trained to match the fused probability distribution, combining the knowledge from all student instances.

### D.2.3 STEP 1: MULTI-INSTANCE STUDENT TRAINING

**Teacher Diversity.** We select  $K$  diverse teacher LLMs  $\{T_{\phi}^{(1)}, T_{\phi}^{(2)}, \dots, T_{\phi}^{(K)}\}$  that differ in:

- **Architecture:** Different model families (e.g., Llama, Qwen, Mistral)
- **Training data:** Models trained on different corpora with varying emphases
- **Size:** Varying parameter counts offering different capacity-quality tradeoffs

**Parallel Pipeline Execution.** For each teacher  $T_{\phi}^{(k)}$ , we execute the complete Stages 1-5 pipeline, producing a corresponding student instance  $S_{\theta}^{(k)}$ :

$$S_{\theta}^{(k)} = \text{Pipeline}_{1-5}(\mathcal{D}_{\text{KGA}}, T_{\phi}^{(k)}, S_{\theta}^{(0)}) \quad (59)$$

where  $S_{\theta}^{(0)}$  is the same base student model used across all instances. Each  $S_{\theta}^{(k)}$  has learned from a different teacher’s perspective.

**Teacher Selection Criteria.** Teachers are selected to maximize diversity while maintaining quality:

- Minimum benchmark performance threshold (e.g.,  $> 60\%$  on held-out validation)
- Low output distribution correlation with other selected teachers
- Complementary strengths across different task categories

#### D.2.4 STEP 2: PROBABILISTIC DISTRIBUTION EXTRACTION

**Shared Evaluation Corpus.** We curate a fusion corpus  $\mathcal{C}_{\text{fusion}}$  from the MCQ dataset, ensuring coverage of all topic areas:

$$\mathcal{C}_{\text{fusion}} = \text{Sample}(\mathcal{D}_{\text{KGA}}, n_{\text{fusion}}, \text{stratified by topic}) \quad (60)$$

**Distribution Matrix Extraction.** For each text sequence  $t \in \mathcal{C}_{\text{fusion}}$ , we extract the probabilistic distribution matrix from each student instance:

$$\mathbf{P}_t^{(k)} \in \mathbb{R}^{|t| \times V} \quad (61)$$

where the  $(i, j)$ -th entry is  $p^{(k)}(t_j | t_{<i})$ -the probability that student  $S_{\theta}^{(k)}$  assigns to token  $t_j$  at position  $i$  given preceding context.

**Token Alignment Across Vocabularies.** When student instances use different tokenizers (due to different base models), we apply minimum edit distance (MinED) alignment Wan et al. (2024a):

$$\text{Align}(\mathbf{P}_t^{(k)}, \mathbf{P}_t^{(l)}) = \text{MinED-Mapping}(\text{tokenize}^{(k)}(t), \text{tokenize}^{(l)}(t)) \quad (62)$$

For tokens without exact vocabulary matches, we map to the closest token by edit distance. This enables fusion across models with different tokenizers.

#### D.2.5 STEP 3: DISTRIBUTION FUSION AND TARGET STUDENT TRAINING

The core of Stage 6 is fusing the probabilistic distributions from multiple student instances into a single training signal.

**Fusion Functions.** We employ two fusion strategies following Wan et al. (2024a):

*Minimum Cross-Entropy (MinCE):* Select the distribution matrix with lowest cross-entropy relative to gold labels:

$$\mathbf{P}_t = \mathbf{P}_t^{(k^*)}, \quad \text{where } k^* = \arg \min_k \text{CE}(\mathbf{P}_t^{(k)}, \mathbf{O}_t) \quad (63)$$

where  $\mathbf{O}_t$  is the one-hot label matrix and  $\text{CE}(\cdot, \cdot)$  is cross-entropy loss.

*Weighted Average Cross-Entropy (AvgCE):* Weighted combination of distribution matrices based on cross-entropy:

$$\mathbf{P}_t = \sum_{k=1}^K w_k \mathbf{P}_t^{(k)}, \quad \text{where } w_k = \frac{\exp(-\text{CE}(\mathbf{P}_t^{(k)}, \mathbf{O}_t)/\tau)}{\sum_j \exp(-\text{CE}(\mathbf{P}_t^{(j)}, \mathbf{O}_t)/\tau)} \quad (64)$$

The temperature  $\tau$  controls the sharpness of the weighting.

**Empirical Preference.** Following Wan et al. (2024a), we default to MinCE as it consistently outperforms AvgCE across benchmarks. AvgCE’s weighted averaging can introduce “noise” from lower-quality predictions.

**Target Student Training.** A new target student  $S_{\theta}^{\text{target}}$  is trained to match the fused distribution through continual training:

$$\mathcal{L}_{\text{fusion}} = -\mathbb{E}_{t \sim \mathcal{C}_{\text{fusion}}} [D_{\text{KL}}(\mathbf{Q}_t \| \mathbf{P}_t)] \quad (65)$$

where  $\mathbf{Q}_t$  is the target student’s distribution matrix and  $\mathbf{P}_t$  is the fused distribution.

**Combined Training Objective.** The target student is trained with a combined objective including standard language modeling:

$$\mathcal{L} = \lambda \mathcal{L}_{\text{CLM}} + (1 - \lambda) \mathcal{L}_{\text{fusion}} \quad (66)$$

where  $\mathcal{L}_{\text{CLM}}$  is the causal language modeling loss and  $\lambda$  balances the two objectives (typically  $\lambda = 0.9$ ).

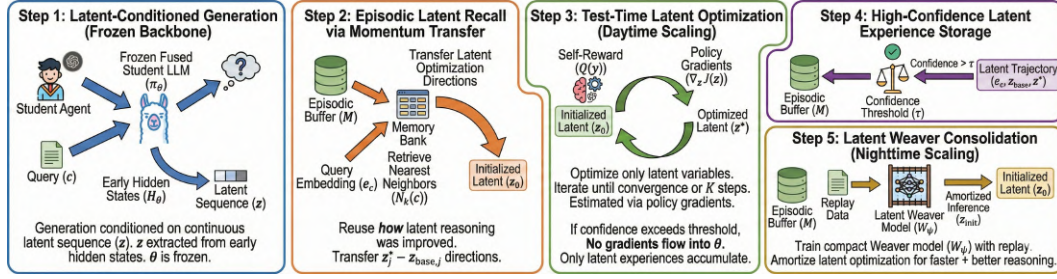


Figure 15: Stage 7: Post-Instructional Reflection

**Algorithm Summary.** The complete FuseLLM-inspired fusion procedure is:

**Require:** Student instances  $\{S_{\theta}^{(1)}, \dots, S_{\theta}^{(K)}\}$ , fusion corpus  $\mathcal{C}_{\text{fusion}}$

- 1: Initialize target student  $S_{\theta}^{\text{target}}$  with one of the source instances
- 2: **for** each text  $t \in \mathcal{C}_{\text{fusion}}$  **do**
- 3:   Extract distribution matrices  $\{\mathbf{P}_t^{(1)}, \dots, \mathbf{P}_t^{(K)}\}$
- 4:   Apply token alignment if necessary
- 5:   Compute fused distribution  $\mathbf{P}_t$  using MinCE (Eq. 63)
- 6:   Update  $S_{\theta}^{\text{target}}$  by minimizing combined loss (Eq. 66)
- 7: **end for**
- 8: **return** Fused target student  $S_{\theta}^{\text{target}}$

### D.3 STAGE 7: POST-INSTRUCTIONAL REFLECTION

Having completed Feedback-Driven Practice (Stage 5) and Multi-Teacher Knowledge Distillation (Stage 6), the student model has acquired diverse knowledge from multiple teaching perspectives. However, a critical challenge remains: the student’s own *reasoning aptitude*—its internal mechanism for generating solutions—may diverge from the heterogeneous patterns absorbed during multi-teacher learning. Stage 7 addresses this through **Post-Instructional Reflection**, a self-evolving test-time scaling methodology that enables the student to align its reasoning processes through latent-space optimization, inspired by recent advances in latent test-time scaling and reflection such as Zhang et al. (2025c).

#### D.3.1 PIPELINE OVERVIEW

Post-Instructional Reflection implements a five-step iterative process that enables the student model to evolve through unsupervised test-time scaling:

1. **Step 1: Latent-Conditioned Generation** - Generation is conditioned on continuous latent sequences extracted from the frozen student backbone, enabling gradient-based steering of the generative process.
2. **Step 2: Episodic Latent Recall via Momentum Transfer** - An episodic buffer stores high-quality optimization experiences; for new queries, relevant experiences are retrieved and their optimization “momentum” is transferred to inform initialization.
3. **Step 3: Test-Time Latent Optimization (Daytime Scaling)** - The latent sequence is iteratively refined through self-supervised policy gradient optimization, guided by a self-rewarding quality function.
4. **Step 4: High-Confidence Latent Experience Storage** - When the optimized output exceeds a confidence threshold, the latent trajectory is archived to the episodic buffer for future retrieval.
5. **Step 5: Latent Weaver Consolidation (Nighttime Scaling)** - Periodically, a compact “latent weaver” model is trained to amortize the latent optimization process, distilling episodic experiences into procedural knowledge.

This pipeline operates **entirely without ground-truth labels**-the student evolves through self-supervised reflection on its own outputs, paralleling how humans refine understanding through practice and introspection.

### D.3.2 STEP 1: LATENT-CONDITIONED GENERATION

The first step reformulates generation as a latent-conditioned process, enabling gradient-based control over the student’s outputs while keeping the backbone frozen.

**Latent-Space Intervention.** Let  $\pi_\theta$  denote the fused student model (from Stage 6) with frozen parameters  $\theta$ . For a given query context  $\mathbf{c}$ , standard generation samples from  $p(\mathbf{y}|\mathbf{c}; \theta)$ . We introduce an auxiliary latent sequence  $\mathbf{z} = (z_1, z_2, \dots, z_{L'}) \in \mathbb{R}^{L' \times d}$  that conditions the generative process:

$$\mathbf{y} \sim p(\mathbf{y}|\mathbf{c}, \mathbf{z}; \theta) \quad (67)$$

The latent sequence  $\mathbf{z}$  acts as a dynamic, instance-specific control signal that modulates generation without modifying the underlying model parameters.

**Base Latent Extraction.** The initial latent sequence is derived from the model’s own hidden states during a preliminary decoding pass. Given query  $\mathbf{c}$ , we perform greedy Chain-of-Thought decoding to obtain the full hidden state trajectory, then extract the first  $L'$  latent vectors as the base initialization:

$$\mathbf{z}_{\text{base}} = H_\theta(\mathbf{c})_{1:L'} \quad (68)$$

where  $H_\theta(\mathbf{c})$  denotes the sequence of hidden states produced during decoding and  $L'$  is a hyperparameter controlling latent sequence length (typically  $L' = 15\text{-}30$ ).

**Latent Integration Mechanisms.** The latent sequence can be integrated into the model’s computation through several mechanisms:

- *Input prepending:* Latent vectors are prepended to the input embedding sequence
- *KV-cache augmentation:* Latent vectors augment the model’s key-value cache
- *Hidden state modulation:* Latent vectors directly modulate intermediate hidden states

The specific mechanism is model-dependent; all enable the core capability of gradient-based latent optimization.

**Cognitive Interpretation.** The base latent sequence represents the student’s “initial reasoning trajectory”-a preliminary approach to the problem encoded in continuous representation space. This trajectory can be refined through optimization to produce better outputs, paralleling how human experts may initially approach a problem one way, then mentally reconsider and improve their approach.

### D.3.3 STEP 2: EPISODIC LATENT RECALL VIA MOMENTUM TRANSFER

The second step leverages an episodic memory buffer to inform latent initialization, enabling the student to benefit from its accumulated optimization experiences.

**Episodic Buffer Structure.** The episodic buffer  $\mathcal{M}$  maintains a collection of high-quality test-time scaling experiences. Each entry is a triplet:

$$\mathcal{M} = \{(\mathbf{e}_{\mathbf{c}_j}, \mathbf{z}_{\text{base},j}, \mathbf{z}_j^*)\}_{j=1}^{|\mathcal{M}|} \quad (69)$$

where  $\mathbf{e}_{\mathbf{c}_j}$  is the semantic embedding of the  $j$ -th query context,  $\mathbf{z}_{\text{base},j}$  is the initial latent sequence and  $\mathbf{z}_j^*$  is the optimized latent sequence that led to high-quality output.

**Associative Retrieval.** Upon receiving a new query  $\mathbf{c}$ , we compute its semantic embedding  $\mathbf{e}_{\mathbf{c}}$  using the frozen model’s final hidden state, then retrieve the top- $k$  most similar experiences from the buffer:

$$\mathcal{N}_k(\mathbf{c}) = \text{TopK}_j\{(\mathbf{e}_{\mathbf{c}_j}, \mathbf{z}_{\text{base},j}, \mathbf{z}_j^*) \in \mathcal{M}\}, \quad \text{based on } S(\mathbf{e}_{\mathbf{c}}, \mathbf{e}_{\mathbf{c}_j}) \quad (70)$$

where  $S(\cdot, \cdot)$  is cosine similarity. This retrieval identifies past problems whose optimization experiences are likely relevant to the current query.

**Weighted Momentum Transfer.** Rather than directly averaging retrieved latent sequences (which may be problem-specific and thus misleading), we transfer the *optimization momentum*—the direction of improvement from base to optimized latent:

$$\Delta \mathbf{z}_j = \mathbf{z}_j^* - \mathbf{z}_{\text{base},j} \quad (71)$$

The informed initialization is computed as a weighted combination of the base latent and transferred momenta:

$$\mathbf{z}_0 = \mathbf{z}_{\text{base}} + \sum_{j \in \mathcal{N}_k(\mathbf{c})} \alpha_j \Delta \mathbf{z}_j, \quad \text{where } \alpha_j \propto \exp(S(\mathbf{e}_{\mathbf{c}}, \mathbf{e}_{\mathbf{c}_j})) \quad (72)$$

The softmax-weighted aggregation ensures that more similar experiences contribute more strongly, while the momentum formulation transfers *how* to improve rather than *what* the final answer was.

**Cognitive Interpretation.** This mechanism parallels human analogical reasoning: when faced with a new problem, experts recall similar problems they have solved before, not to copy the solution directly, but to apply the same *approach* or *insight* that led to success. The momentum transfer captures this “approach” in continuous representation space.

#### D.3.4 STEP 3: TEST-TIME LATENT OPTIMIZATION (DAYTIME SCALING)

The third step performs fast, query-specific optimization of the latent sequence to maximize output quality, corresponding to the “daytime” phase of CLS-inspired evolution.

**Self-Rewarding Quality Function.** Since ground-truth labels are unavailable, optimization is guided by a self-rewarding quality function  $Q(\mathbf{y})$  that evaluates the generated output. Following established practices in self-rewarding language models Yuan et al. (2024), the student model itself serves as the evaluator, assigning normalized scores across multiple criteria:

$$Q(\mathbf{y}) = \frac{1}{7} \left( s_{\text{ans}}(\mathbf{y}) + s_{\text{comp}}(\mathbf{y}) + s_{\text{calc}}(\mathbf{y}) + 2 s_{\text{form}}(\mathbf{y}) + 2 s_{\text{clar}}(\mathbf{y}) \right) \quad (73)$$

where:

- $s_{\text{ans}}$ : Perceived correctness of the final answer
- $s_{\text{comp}}$ : Accuracy of problem comprehension
- $s_{\text{calc}}$ : Validity of numerical/logical calculations
- $s_{\text{form}}$ : Conformity to expected output format
- $s_{\text{clar}}$ : Clarity and explicitness of the response

The weighting scheme emphasizes format fidelity and clarity, as these are more reliably assessable through self-evaluation.

**Policy Gradient Optimization.** The optimization objective is to find the latent sequence  $\mathbf{z}^*$  that maximizes expected quality:

$$\mathbf{z}^* = \arg \max_{\mathbf{z}} J(\mathbf{z}), \quad \text{where } J(\mathbf{z}) = \mathbb{E}_{\mathbf{y} \sim p(\mathbf{y}|\mathbf{c}, \mathbf{z}; \theta)} [Q(\mathbf{y})] \quad (74)$$

Since the expectation is over discrete samples, we estimate the gradient via the REINFORCE policy gradient estimator:

$$\nabla_{\mathbf{z}} J(\mathbf{z}) \approx \frac{1}{M} \sum_{m=1}^M Q(\mathbf{y}^{(m)}) \nabla_{\mathbf{z}} \log p(\mathbf{y}^{(m)}|\mathbf{c}, \mathbf{z}; \theta) \quad (75)$$

where  $\{\mathbf{y}^{(m)}\}_{m=1}^M$  are  $M$  samples drawn from the current latent-conditioned distribution.

**Iterative Refinement Protocol.** The latent sequence is updated through gradient ascent:

$$\mathbf{z}_{k+1} \leftarrow \mathbf{z}_k + \eta \nabla_{\mathbf{z}_k} J(\mathbf{z}_k) \quad (76)$$

Refinement terminates when: (1) the maximum iteration count  $K$  is reached (typically  $K = 10$ ), or (2)  $\mathbb{E}[Q(\mathbf{y}_k)]$  fails to improve for three consecutive iterations. The final optimized latent sequence  $\mathbf{z}^*$  guides generation of the output  $\mathbf{y}$ .

**Cognitive Interpretation.** Daytime scaling corresponds to the fast, episodic learning that occurs during active problem-solving. The student iteratively refines its approach to a specific problem, using self-assessment to guide improvement. This parallels deliberate practice, where immediate feedback drives incremental refinement.

#### D.3.5 STEP 4: HIGH-CONFIDENCE LATENT EXPERIENCE STORAGE

The fourth step selectively archives high-quality optimization experiences to the episodic buffer, ensuring that only valuable experiences inform future adaptation.

**Confidence-Based Archival.** Not all optimization trajectories are equally valuable. Following the principle that memory consolidation should prioritize high-quality experiences, we archive a triplet  $(\mathbf{e}_c, \mathbf{z}_{\text{base}}, \mathbf{z}^*)$  to the buffer  $\mathcal{M}$  only if the final expected quality exceeds threshold  $\tau$ :

$$\mathbb{E}[Q(\mathbf{y}^*)] \geq \tau \implies (\mathbf{e}_c, \mathbf{z}_{\text{base}}, \mathbf{z}^*) \rightarrow \mathcal{M} \quad (77)$$

A typical threshold is  $\tau = 0.5$ , corresponding to moderate confidence that the output is correct and well-formed.

**No Gradient Flow to Backbone.** A critical design choice is that latent optimization **does not propagate gradients to the backbone parameters**  $\theta$ . Only the latent experiences accumulate in memory; the student’s core knowledge remains frozen. This separation ensures that:

- The backbone preserves the knowledge acquired during Stages 1-6
- Latent optimization cannot cause catastrophic forgetting
- The episodic buffer captures modular, transferable “skills” rather than parameter changes

**Buffer Management.** As the buffer grows, we employ recency-weighted sampling for retrieval and periodic pruning of low-similarity entries. This maintains buffer quality while bounding memory requirements.

**Cognitive Interpretation.** This step parallels the hippocampal encoding of episodic memories: specific problem-solving experiences are stored with sufficient context (query embedding) to enable future retrieval. The confidence threshold corresponds to the brain’s tendency to consolidate salient, successful experiences more strongly than routine or failed attempts.

#### D.3.6 STEP 5: LATENT WEAVER CONSOLIDATION (NIGHTTIME SCALING)

The fifth step performs slow, procedural consolidation of accumulated episodic knowledge into a compact model, corresponding to the “nighttime” phase of CLS-inspired evolution.

**The Latent Weaver.** While daytime scaling excels at query-specific adaptation, its knowledge remains fragmented within the episodic buffer. To achieve generalizable improvement, we introduce the *latent weaver*  $\mathbf{W}_\psi$ : a compact model trained to predict optimized latent sequences from query embeddings and base latents:

$$\mathbf{W}_\psi(\mathbf{e}_c, \mathbf{z}_{\text{base}}) \approx \mathbf{z}^* \quad (78)$$

The weaver is instantiated as a small LLM (e.g., Qwen-2.5-1.5B), enabling it to capture complex patterns while remaining computationally efficient.

**Consolidation Through Experience Replay.** Periodically (after every  $T$  test instances, typically  $T = 200$ ), the latent weaver is trained on accumulated buffer experiences. The training objective minimizes reconstruction error between the weaver’s predictions and archived optimized latents:

$$\mathcal{L}(\psi) = \mathbb{E}_{(\mathbf{e}_{c_j}, \mathbf{z}_{\text{base},j}, \mathbf{z}_j^*) \sim \mathcal{M}} \left[ \|\mathbf{W}_\psi(\mathbf{e}_{c_j}, \mathbf{z}_{\text{base},j}) - \mathbf{z}_j^*\|_2^2 \right] \quad (79)$$

This training “weaves” the sparse, episodic optimization experiences into the continuous parametric space of the weaver, distilling procedural intuition that can generate superior initial latent representations for future queries.

**Dual-Phase Evolution Cycle.** The complete evolution process alternates between daytime and nighttime phases:

$$(\mathcal{M}, \mathbf{W}_\psi) \xrightarrow{\Phi_{\text{day}}} \mathcal{M}' \xrightarrow{\Phi_{\text{night}}} (\mathcal{M}', \mathbf{W}'_\psi) \quad (80)$$

where  $\Phi_{\text{day}}$  denotes daytime optimization (Steps 1-4) that updates the buffer and  $\Phi_{\text{night}}$  denotes nighttime consolidation (Step 5) that updates the weaver. After nighttime consolidation, subsequent queries benefit from the weaver’s improved initialization:

$$\mathbf{z}'_{\text{base}} = \mathbf{W}_\psi(\mathbf{e}_c, \mathbf{z}_{\text{base}}) \quad (81)$$

This transformed base latent incorporates procedural insights from prior nighttime consolidations, providing a superior starting point for daytime optimization.

**Cognitive Interpretation.** Nighttime scaling mirrors the neocortical consolidation that occurs during sleep, when the brain integrates episodic memories into generalizable schemas. The latent weaver learns abstract patterns-“how to optimize latents for this type of problem”-rather than memorizing specific solutions. This procedural knowledge transfers across problems, enabling faster and more effective optimization.

## E PHASE III : TEACHING

### E.1 STAGE 9: LEARNING BY TEACHING

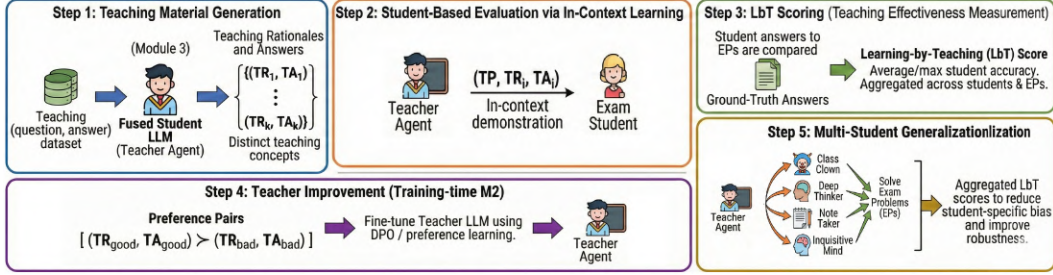


Figure 16: Stage 9: Learning by Teaching

“I couldn’t reduce it to the freshman level.  
That means we really don’t understand it.”

-Richard Feynman

#### E.1.1 MOTIVATION

Stage 9 represents the culmination of the LMT<sup>2</sup> framework, where the mastered student transitions to a teacher role. This stage is grounded in the well-established **protégé effect** Roscoe & Chi (2007); Biswas et al. (2005): teaching others consolidates and deepens the teacher’s own understanding. As Feynman’s quote suggests, the ability to explain concepts simply serves as a litmus test for true comprehension. Related works include Zhang et al. (2025d); Dinucu-Jianu et al. (2025); Shi et al. (2025); Ning et al. (2024a); Liu et al. (2024); An et al. (2023)

The cognitive basis for Learning by Teaching (LbT) operates at three levels Biswas et al. (2005); Chi et al. (2001):

- **L1: Observing Feedback** - The teacher instructs students, who provide feedback (e.g., exam scores, clarifying questions).
- **L2: Learning from Feedback** - Based on feedback, the teacher analyzes which concepts students (mis)understood, improving teaching strategy and deepening their own understanding.
- **L3: Iterative Refinement** - The teacher iteratively teaches, observes feedback (L1) and learns from it (L2), progressively refining both teaching materials and internal knowledge representations.

This framework bridges human cognition and AI reasoning by implementing the Feynman learning method in LLMs, enabling *weak-to-strong generalization*-stronger models can improve by teaching weaker models, without requiring human-generated data or supervision from more capable systems.

#### E.1.2 PIPELINE OVERVIEW

The LbT pipeline consists of five interconnected steps, illustrated in Figure 16.

#### E.1.3 STEP 1: TEACHING MATERIAL GENERATION

The Teacher LLM samples a Teaching Problem (TP) from a curated dataset and generates multiple candidate Teaching Rationales (TR) and Teaching Answers (TA). Formally:

$$\{(\text{TR}_i, \text{TA}_i)\}_{i=1}^N \sim \mathcal{M}_{\text{teacher}}(\cdot \mid \text{TP}) \quad (82)$$



where  $N$  is the number of candidate teaching pairs generated per problem. The key insight is that different rationales may employ distinct problem-solving strategies, analogies, or levels of detail-enabling downstream evaluation of which pedagogical approaches are most effective.

**Implementation:** We use temperature-based sampling ( $T > 0$ ) to ensure diversity among generated rationales. Each rationale-answer pair represents a different “teaching style” that will be evaluated in subsequent steps.

#### E.1.4 STEP 2: STUDENT-BASED EVALUATION VIA IN-CONTEXT LEARNING

Each teaching pair  $(TP, TR_i, TA_i)$  is used as an in-context learning (ICL) demonstration for Student LLMs. Students then attempt to solve similar Exam Problems (EPs):

$$(ER_j, EA_j) \sim \mathcal{M}_{\text{student}}(\cdot \mid [TP, TR_i, TA_i], EP_j) \quad (83)$$

where  $ER_j$  and  $EA_j$  are the student’s Exam Rationale and Exam Answer for problem  $j$ . The critical assumption is that *exam problems share similar problem-solving strategies with the teaching problem*, enabling transfer of the demonstrated reasoning approach.

**Cognitive Alignment:** This mirrors human learning where students learn from worked examples and apply similar strategies to novel but related problems. The ICL mechanism enables students to “learn” the logic embedded in teaching materials without explicit gradient updates.

#### E.1.5 STEP 3: LbT SCORING (TEACHING EFFECTIVENESS MEASUREMENT)

Student performance on exam problems provides a signal for teaching effectiveness. The LbT Score aggregates correctness across students and exam problems:

$$\text{LbT}(TR_i, TA_i) = \frac{1}{|\mathcal{S}| \cdot |\mathcal{E}|} \sum_{s \in \mathcal{S}} \sum_{e \in \mathcal{E}} \mathbb{I}[\text{EA}_{s,e} = \text{GT}_e] \quad (84)$$

where  $\mathcal{S}$  is the set of student models,  $\mathcal{E}$  is the set of exam problems and  $\text{GT}_e$  is the ground-truth answer for exam problem  $e$ .

**Key Insight:** Teaching materials that make it easier for students to learn exhibit *clearer and more accurate logic*. This LbT-TMQ (Teaching Material Quality) assumption underlies the scoring mechanism: better rationales lead to higher student accuracy, which in turn indicates higher teaching quality.

#### E.1.6 STEP 4: TEACHER IMPROVEMENT VIA PREFERENCE LEARNING

LbT scores are used to construct preference pairs for fine-tuning the Teacher LLM:

$$(TR_{\text{good}}, TA_{\text{good}}) \succ (TR_{\text{bad}}, TA_{\text{bad}}) \iff \text{LbT}_{\text{good}} > \text{LbT}_{\text{bad}} \quad (85)$$

These preference pairs are used to fine-tune the teacher via Direct Preference Optimization (DPO) Rafailov et al. (2023):

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[ \log \sigma \left( \beta \log \frac{\pi_{\theta}(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_{\theta}(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right] \quad (86)$$

where  $y_w$  and  $y_l$  are the preferred and dispreferred teaching responses, respectively.

**Cognitive Alignment:** This implements Level 2 (L2) learning-the teacher internalizes feedback about which teaching strategies were effective, improving its inherent pedagogical ability rather than just selecting better outputs.

### E.1.7 STEP 5: MULTI-STUDENT GENERALIZATION

To ensure robust teaching improvement, we deploy diverse student personas inspired by the SimClass framework Zhang et al. (2024b):

- **Class Clown:** Engages with humor, may highlight when explanations are overly complex or boring.
- **Deep Thinker:** Challenges assumptions, raises counterexamples, tests edge cases.
- **Note Taker:** Summarizes and organizes information, revealing if structure is clear.
- **Inquisitive Mind:** Asks clarifying questions, exposing gaps in explanations.

Each persona is implemented via system prompts that configure the student LLM’s behavior and knowledge level. Aggregating LbT scores across diverse students reduces bias from any single student type:

$$\text{LbT}_{\text{agg}}(\text{TR}_i, \text{TA}_i) = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \text{LbT}_p(\text{TR}_i, \text{TA}_i) \quad (87)$$

where  $\mathcal{P}$  is the set of student personas.

## E.2 ALGORITHM SUMMARY

---

### Algorithm 4 Learning by Teaching (LbT) Training

---

**Require:** Teacher model  $\mathcal{M}_T$ , Student models  $\{\mathcal{M}_S^p\}_{p \in \mathcal{P}}$ , Teaching dataset  $\mathcal{D}_{\text{TP}}$ , Exam dataset  $\mathcal{D}_{\text{EP}}$

- 1: **for** each Teaching Problem  $\text{TP} \in \mathcal{D}_{\text{TP}}$  **do**
  - 2:   Generate  $N$  teaching pairs:  $\{(\text{TR}_i, \text{TA}_i)\}_{i=1}^N \sim \mathcal{M}_T(\cdot \mid \text{TP})$
  - 3:   **for** each teaching pair  $(\text{TR}_i, \text{TA}_i)$  **do**
  - 4:     **for** each student persona  $p \in \mathcal{P}$  **do**
  - 5:       **for** each Exam Problem  $\text{EP}_j \in \mathcal{D}_{\text{EP}}$  similar to  $\text{TP}$  **do**
  - 6:         Student solves:  $\text{EA}_{i,p,j} \sim \mathcal{M}_S^p(\cdot \mid [\text{TP}, \text{TR}_i, \text{TA}_i], \text{EP}_j)$
  - 7:       **end for**
  - 8:     **end for**
  - 9:     Compute  $\text{LbT}_i = \frac{1}{|\mathcal{P}| \cdot |\mathcal{E}|} \sum_{p,j} \mathbb{1}[\text{EA}_{i,p,j} = \text{GT}_j]$
  - 10:   **end for**
  - 11:   Construct preference pairs:  $(\text{TR}_{\text{best}}, \text{TA}_{\text{best}}) \succ (\text{TR}_{\text{worst}}, \text{TA}_{\text{worst}})$
  - 12: **end for**
  - 13: Fine-tune  $\mathcal{M}_T$  using DPO on collected preference pairs
  - 14: **return** Improved Teacher model  $\mathcal{M}'_T$
-